Information Theory: Surprise, Compression, and Communication

Mick Cooney

14 October 2026

Table of Contents

1 Orientation and Notation **3**
- 1.1 Purpose, Scope, and Background . . . 3
- 1.2 Reading Route . . . 4
- 1.3 Notation . . . 5
- 1.4 Units, Supports, and Claims . . . 6

2 Probability Foundations **6**
- 2.1 Sample Spaces and Random Variables . . . 7
- 2.1.1 Joint and Marginal Distributions . . . 7
- 2.2 Conditional Distributions and Bayes' Rule . . . 8
- 2.2.1 Independence and Factorisation . . . 9
- 2.3 Expectation and Conditional Expectation . . . 9
- 2.3.1 Conditional Independence and Markov Chains . . . 10
- 2.4 Continuous Variables: Probability Is Mass, Density Is Not . 10
- 2.5 Base R and Python: Inspect a Joint Law . . . 11

3 Entropy and Divergences **13**
- 3.1 Surprise as Log Probability . . . 13
- 3.2 Shannon Entropy . . . 13
- 3.2.1 Joint Entropy and Chain Rule . . . 15
- 3.2.2 Conditioning Reduces Entropy . . . 16
- 3.3 KL Divergence . . . 16
- 3.3.1 Cross-Entropy and Log Loss . . . 17
- 3.4 Entropy of a Bernoulli Variable . . . 18
- 3.4.1 Continuous Entropy and Relative Entropy . . . 18
- 3.5 Base R and Python Calculations . . . 20

4 Information Between Variables **21**
- 4.1 Conditional Entropy as Remaining Uncertainty . . . 22
- 4.2 Mutual Information . . . 22
- 4.2.1 Conditional Mutual Information and Chain Rules . . . 24

4.3 Independence, Synergy, and Interaction . . . 25
4.4 Data-Processing Inequality . . . 25
4.5 Continuous Variables and Mutual Information . . . 27
4.6 Base R and Python Calculation From a Joint Table . . . 28

5 Lossless Source Coding 29
5.1 From Probabilities to Code Lengths . . . 29
5.1.1 Kraft Inequality . . . 29
5.1.2 The Entropy Lower Bound . . . 30
5.2 Shannon Code and the One-Bit Redundancy Bound . . . 31
5.3 Huffman Coding and a Complete Example . . . 31
5.3.1 Base R and Python for Expected Length . . . 33
5.4 Block Coding and the Noiseless Source-Coding Theorem . . . 34
5.4.1 What “lossless” Means in Practice . . . 35
5.5 Arithmetic Coding: Probability Intervals . . . 36

6 Noisy Channels and Capacity 37
6.1 A Channel as a Conditional Distribution . . . 37
6.2 Binary Symmetric Channel: One Input Law, One Capacity . . . 38
6.2.1 Binary Erasure Channel . . . 40
6.3 Asymmetric Channels and Input Optimisation . . . 41
6.4 Capacity and the Channel Coding Theorem . . . 42
6.4.1 Error Correction as Geometry . . . 45
6.5 The Real AWGN Channel . . . 46

7 Lossy Compression and Rate-Distortion 47
7.1 Distortion and the Rate–Distortion Function . . . 47
7.2 Binary Hamming Distortion . . . 49
7.3 Gaussian Squared-Error Distortion . . . 50
7.4 Choosing a Distortion Measure . . . 51

8 Estimating Information from Data 52
8.1 Plug-In Estimates and Their Bias . . . 53
8.2 Joint Tables, Sparse Cells, and Mutual Information . . . 54
8.3 Permutation Tests for Dependence . . . 58
8.3.1 Bootstrap Uncertainty and Resampling Units . . . 58
8.4 Continuous Variables and Estimator Choice . . . 59
8.4.1 Reporting an Estimate so It Can Be Reused . . . 60
8.4.2 A Practical Estimation Workflow . . . 61

9 Connections to Statistics and Machine Learning 61
9.1 Log Loss, Cross-Entropy, and KL Direction . . . 62
9.1.1 Calibration Is Not the Same as Discrimination . . . 63
9.2 Evidence, Features, and Predictive Information . . . 63
9.3 The Information Bottleneck . . . 64
9.3.1 Evidence Is Not Causation . . . 65

9.3.2 Privacy: Information Measures and Guarantees 66
9.4 A Practical Quantity-Selection Guide 67

10 Exercises **69**
10.1 Exercise 1: Probability and Labelling 69
10.2 Exercise 2: KL Direction and Support 69
10.3 Exercise 3: Pairwise Independence and Parity 69
10.4 Exercise 4: Equality in Data Processing 70
10.5 Exercise 5: Kraft and a Code Assignment 70
10.6 Exercise 6: Fractional Entropy and Integer Lengths 70
10.7 Exercise 7: Input Bias on a BSC 70
10.8 Exercise 8: Erasures and Substitutions 71
10.9 Exercise 9: A Z-Channel Optimisation 71
10.10 Exercise 10: Coding Rate and Error Probability 71
10.11 Exercise 11: Choose a Distortion Measure 72
10.12 Exercise 12: Gaussian Rate–Distortion 72
10.13 Exercise 13: Entropy Bias and a Sparse Table 73
10.14 Exercise 14: KL Direction and Prediction 73
10.15 Exercise 15: Information Bottleneck and Causation 73
10.16 Exercise 16: Select a Quantity for a System Claim 74

11 Glossary **74**

12 Further Reading **79**

1 Orientation and Notation

The sections that follow build from probability models to information quantities and then to coding consequences. This orientation fixes the scope, reading route, and notation used throughout, so later formulas can be read against a clearly stated model and unit.

1.1 Purpose, Scope, and Background

Information theory gives precise ways to talk about uncertainty, dependence, representation, and communication. Its central move is to describe a source by a probability law, then ask how that law constrains what can be inferred or encoded. The word *information* here does not mean semantic importance, truth, usefulness, or the number of bytes in a file. Those may matter in an application, but they are not what entropy measures.

The organising questions are: how uncertain is the next outcome before it ar-

rives; how much does an observation change that uncertainty; and how many bits are needed to represent outcomes under a stated lossless coding rule? These questions connect probability to compression. The same definitions later support noisy communication, privacy analysis, statistical modelling, and machine learning, but those applications need additional assumptions.

This primer develops the finite-alphabet foundations and the main regular continuous-variable cautions. It is an advanced technical primer rather than a measure-theoretic probability text, a complete coding-theory course, or a communications-engineering handbook. The emphasis is on knowing which quantity answers which question and what hypotheses make a theorem true.

You should know algebra, logarithms, introductory probability, and enough calculus to follow expectations and elementary inequalities. Basic familiarity with R vectors, matrices, and functions is useful for running the examples, but the mathematical explanations do not require programming. Examples are available in both R and Python, with R as the primary presentation and the default HTML tab. The R examples use base R and require no additional packages; the Python examples use the standard library except for the empirical mutual-information calculation, which requires NumPy. Both implementations remain visible in PDF and print. Each is a direct calculation intended to expose conventions and assumptions, not a production compressor. Run the code blocks in order within each section; later R estimation examples reuse the `mi_bits()` function defined in that section.

1.2 Reading Route

Start with probability if conditional laws and expectations are unfamiliar. Entropy and divergence describe one distribution or compare two. Mutual information then treats dependence, source coding turns a known source law into a representation, and channel coding asks what a noisy link can convey. The final technical chapters address reconstruction under a distortion budget, estimation from finite data, and statistical applications. The rendered editions include a contents list; each chapter defines its local model so that you can also consult it independently.

Source coding removes statistical redundancy, whereas channel coding adds structured redundancy to withstand noise. Both are asymptotic theories under stated models; the worked examples and exercises show how to calculate finite

quantities without confusing them with theorem guarantees.

1.3 Notation

Unless a section explicitly says otherwise, logarithms are base 2 and information is measured in bits. The convention $0 \log_2 0 = 0$ is used in entropy sums by continuity. For a countable distribution, formulas require the relevant sums and expectations to exist; finiteness of the alphabet is a sufficient, convenient assumption for most identities below.

Symbol	Meaning	First Use
X, Y, Z	Random variables; lowercase letters denote their values	Probability spaces
$\mathcal{X}$	Alphabet or state space of X	Discrete variables
$p_X(x)$, $p(x)$	Probability mass function, or density when explicitly stated	Probability spaces
$p_{XY}(x, y)$	Joint mass function	Joint and conditional laws
$p(x \mid y)$	Conditional mass function on events with $p_Y(y) > 0$	Conditional laws
$\mathbb{E}[X]$	Expectation under the stated distribution	Expectation
$H(X)$, $H(X \mid Y)$	Discrete entropy and conditional entropy, in bits	Entropy
$D_{\mathrm{KL}}(P\|Q)$	KL divergence of P relative to Q	Divergence
$I(X;Y)$, $I(X;Y \mid Z)$	Mutual information and conditional mutual information	Dependence
$\ell(x)$, L	Codeword length for x and expected length	Source coding

Symbol	Meaning	First Use
n	Block length, stated locally	Block coding
ε	Positive tolerance or error probability, declared by context	Theorems

For a discrete random variable, $p_X(x) = P(X = x)$. We often omit the subscript when the variable is unambiguous. For continuous variables, p_X will be called a density and integrals replace sums; a density value is not a probability. We use P and Q for probability distributions when discussing divergence, and p and q for their mass functions or densities. A conditional law is only defined up to events of probability zero; finite-alphabet formulas can assign arbitrary conditional distributions on zero-probability conditioning events without changing any expectation.

1.4 Units, Supports, and Claims

Changing all logarithms from base 2 to natural logarithms changes bits to nats by multiplication by $\ln 2$. Inequalities and minimisers are unaffected by this fixed positive scale. Keep the base explicit when comparing numerical values.

The support of a discrete distribution P is $\operatorname{supp}(P) = \{x : P(x) > 0\}$. A formula involving $\log(P(x)/Q(x))$ is evaluated only where $P(x) > 0$. If $P(x) > 0$ but $Q(x) = 0$, the KL divergence is $+\infty$. This support convention prevents the common but invalid operation of evaluating $\log 0$ and hoping a zero coefficient will cancel it.

The information quantities in Sections 3 and 4 depend on a probability model. In an empirical study, that model may be known, estimated, or implicitly replaced by empirical frequencies. State which case applies. A measured code length under a model is not automatically the entropy of the data-generating process.

2 Probability Foundations

Information quantities are expectations or comparisons under probability laws, so their interpretation depends on specifying the random variables and their

joint distribution. This section establishes the sample-space, marginal, conditional, and expectation rules used in the later definitions.

2.1 Sample Spaces and Random Variables

A probability model consists of a sample space Ω, a collection of events, and a probability measure P. A random variable X maps an outcome in Ω to a value in a measurable state space. For the finite examples in this section, the event structure is just the set of subsets of Ω, so measure-theoretic details can remain in the background.

A discrete random variable has a probability mass function (pmf) $p_X(x) = P(X = x)$ satisfying $p_X(x) \geq 0$ and $\sum_{x \in \mathcal{X}} p_X(x) = 1$. The alphabet $\mathcal{X}$ lists possible values, including values whose probability may be zero. A random variable is not its realised value: X names the uncertain quantity, while x is one possible outcome.

For example, let X be the result of a biased coin toss with $P(X = \mathrm{H}) = 0.4$ and $P(X = \mathrm{T}) = 0.6$. The event $\{X = \mathrm{H}\}$ has probability 0.4. The labels H and T could be replaced by 0 and 1 without changing any information quantity, provided the probabilities travel with the labels.

2.1.1 Joint and Marginal Distributions

For two discrete variables X and Y, the joint pmf $p_{XY}(x, y) = P(X = x, Y = y)$ is non-negative and sums to one over pairs. Marginalisation removes a variable by summing over its possible values:

$$p_X(x) = \sum_y p_{XY}(x, y), \qquad p_Y(y) = \sum_x p_{XY}(x, y).$$

The table below gives a joint law for two fair bits that agree with probability 0.8. It is one compact way to specify both marginals and dependence.

$p_{XY}(x, y)$	$y = 0$	$y = 1$	Row sum
$x = 0$	0.4	0.1	0.5
$x = 1$	0.1	0.4	0.5
Column sum	0.5	0.5	1

Each marginal is fair. The variables are nevertheless dependent because $p_{XY}(0,0) = 0.4$ differs from $p_X(0)p_Y(0) = 0.25$. Marginal distributions alone do not specify a joint law; dependence lives in the way the variables occur together.

For several variables, $p_{XYZ}(x,y,z)$ is the joint law, and all lower-dimensional marginals follow by summing over omitted coordinates. An ordered tuple $(X_1, \dots, X_n)$ is itself a random variable with alphabet $\mathcal{X}_1 \times \cdots \times \mathcal{X}_n$. This viewpoint makes block entropy and block coding ordinary single-variable questions over a larger alphabet.

2.2 Conditional Distributions and Bayes' Rule

For $p_Y(y) > 0$, the conditional pmf of X given $Y = y$ is

$$p(x \mid y) = P(X = x \mid Y = y) = \frac{p_{XY}(x,y)}{p_Y(y)}.$$

It is a distribution over x for each fixed y. Rearranging gives the product rule $p_{XY}(x,y) = p_Y(y)p(x \mid y)$. Applying the same identity in the other direction gives Bayes' rule:

$$p(x \mid y) = \frac{p(y \mid x)p_X(x)}{p_Y(y)}, \qquad p_Y(y) = \sum_x p(y \mid x)p_X(x).$$

The distribution p_X is often called a prior, $p(y \mid x)$ a likelihood, and $p(x \mid y)$ a posterior. These names describe roles in an inference calculation; they do not alter the probability identities. The denominator averages the likelihood over the prior and ensures the posterior sums to one.

Worked Example 2.1 (A noisy observation). Let a bit X be equally likely to be 0 or 1. An observation Y flips it independently with probability 0.1. Then $p(Y = 1 \mid X = 0) = 0.1$ and $p(Y = 1 \mid X = 1) = 0.9$. The marginal probability of $Y = 1$ is $0.1(0.5) + 0.9(0.5) = 0.5$. Bayes' rule gives $p(X = 1 \mid Y = 1) = 0.9(0.5)/0.5 = 0.9$. A posterior needs the observation model and the source distribution; the likelihood alone is not the posterior.

2.2.1 Independence and Factorisation

Variables X and Y are independent if their joint distribution factorises:

$$p_{XY}(x, y) = p_X(x)p_Y(y) \quad \text{for every } x, y.$$

Where $p_Y(y) > 0$, this is equivalent to $p(x \mid y) = p_X(x)$. Independence is a property of the joint law, not a visual property of labels or a statement that two variables have no conceivable causal relation. Statistical independence does not, by itself, establish or rule out causation.

For n variables, mutual independence means the full joint law factors into the product of all marginals. Pairwise independence is weaker when $n \geq 3$. For example, let X and Y be independent fair bits and set $Z = X \oplus Y$. Every pair among X, Y, Z is independent, yet the triple is constrained by $X \oplus Y \oplus Z = 0$. When a theorem assumes independence, check whether it requires mutual independence of the whole sequence rather than pairwise independence.

2.3 Expectation and Conditional Expectation

For a discrete variable with finite support, the expectation of a function $g(X)$ is

$$\mathbb{E}[g(X)] = \sum_x p_X(x)g(x).$$

Expectation is an average under a probability law. It is not a claim that an observed finite sample will equal that average. For a joint distribution, $\mathbb{E}[g(X, Y)] = \sum_{x,y} p_{XY}(x, y)g(x, y)$.

The conditional expectation given $Y = y$ is the average over the conditional law, $\mathbb{E}[g(X) \mid Y = y] = \sum_x p(x \mid y)g(x)$. Averaging these conditional expectations over Y recovers the unconditional expectation:

$$\mathbb{E}[g(X)] = \sum_y p_Y(y)\mathbb{E}[g(X) \mid Y = y].$$

This is the law of total expectation. Entropy's conditional form uses exactly this averaging operation: calculate uncertainty separately for each observed value, then average according to how often that value occurs.

2.3.1 Conditional Independence and Markov Chains

The variables X and Z are conditionally independent given Y if, for every y with positive probability,

$$p(x, z \mid y) = p(x \mid y)p(z \mid y).$$

Equivalently, $p(z \mid x, y) = p(z \mid y)$ wherever the conditioning events have positive probability. We write $X \to Y \to Z$ for this Markov condition. The arrows encode a factorisation of a joint distribution, often $p(x, y, z) = p_X(x)p(y \mid x)p(z \mid y)$; they do not assert time order or causation unless those interpretations are independently justified.

This condition is the hypothesis behind the data-processing inequality in Data-Processing Inequality. Merely naming variables in a sequence does not make them a Markov chain. A representation $Z = f(Y)$ is conditionally independent of X given Y because it is determined by Y, but a representation that also uses X directly may violate the condition.

Conditional independence must not be confused with ordinary independence. Suppose Y is a fair bit, and conditional on Y we draw X and Z independently, each equalling Y with probability 0.9. Given $Y = 0$, both X and Z favour zero; given $Y = 1$, both favour one. They satisfy $X \perp Z \mid Y$ by construction. Marginally, however, observing X tells us something about Y and therefore about Z, so X and Z are dependent. The probability that they agree is $0.9^2 + 0.1^2 = 0.82$, greater than the 0.5 agreement probability of independent fair bits.

The reverse implication also fails. Let X and Z be independent fair bits and set $Y = X \oplus Z$. Unconditionally X and Z are independent. Once $Y = 0$ is known, they must be equal; once $Y = 1$ is known, they must differ. The conditional law does not factorise. This is the same parity structure used later to show why conditioning can increase mutual information. The two examples are useful checks when deciding whether a graphical arrow represents a valid factorisation: work from the joint law, not the apparent order of variables.

2.4 Continuous Variables: Probability Is Mass, Density Is Not

A continuous random variable is described by a probability density function (pdf) f_X when one exists. Probabilities are integrals, for example $P(a < X \leq b) =$

$\int_a^b f_X(x)\,dx$. A density can exceed one and has units inverse to those of X; the probability of a single point is ordinarily zero. Consequently, substituting a density value into a discrete pmf formula without changing sums to integrals is a category error.

For jointly continuous variables, the joint density f_{XY} and conditional density $f_{X|Y}$ obey integral analogues of the marginal and product rules. Mixed discrete-continuous laws require sums and integrals over their respective components. A fully general treatment uses probability measures and Radon–Nikodym derivatives; no claim here assumes that every distribution has a density.

2.5 Base R and Python: Inspect a Joint Law

The same joint table can be represented as a matrix whose rows index X and columns index Y. R fills matrices by column unless `byrow = TRUE` is supplied; here that argument keeps the entries in the row order of the displayed table. The code checks that the entries are finite, non-negative, and sum to one, then computes marginals with `rowSums()` and `colSums()`.

R

```r
joint <- matrix(c(0.4, 0.1,
                  0.1, 0.4),
                nrow = 2, byrow = TRUE)

stopifnot(all(is.finite(joint)), all(joint >= 0),
          abs(sum(joint) - 1) < 1e-12)
row_marginals <- rowSums(joint)
column_marginals <- colSums(joint)
row_marginals
column_marginals
```

Python

```python
from math import isfinite

joint = [[0.4, 0.1],
         [0.1, 0.4]]

assert joint and joint[0]
assert all(len(row) == len(joint[0]) for row in joint)
assert all(isfinite(p) and p >= 0 for row in joint for p in row)
row_marginals = [sum(row) for row in joint]
column_marginals = [sum(column) for column in zip(*joint)]
assert abs(sum(row_marginals) - 1.0) < 1e-12
print(row_marginals, column_marginals)
```

Both marginals are `c(0.5, 0.5)`, matching the fair-bit distributions in the table.

The probability rules above are enough to begin calculating information, but a calculation is meaningful only after the random experiment has been spe-

cified. A joint table from repeated independent trials describes a different experiment from a single trajectory of a time series. If observations share a latent cause, treating their rows as independent can give apparently precise frequencies while understating uncertainty about the population law. Information identities still hold for the population distribution; what changes is whether the table in hand estimates it reliably.

Worked example (two noisy stages). Let a fair bit X pass through two independent binary flip mechanisms. The first flips with probability a, producing Y; the second flips with probability b, producing Z. Given Y, the second flip is independent of X, so $X \to Y \to Z$ is a Markov chain. The final bit differs from the initial bit precisely when one stage flips and the other does not. Its effective crossover probability is

$$c = a(1-b) + (1-a)b = a + b - 2ab.$$

For $a = 0.1$ and $b = 0.2$, we obtain $c = 0.26$. If both mechanisms are independent across uses as well as from each other, the composite link is a binary symmetric memoryless channel. If the second flip probability depends on the first flip after conditioning on Y, the simple composition formula need not hold. The calculation is a reminder that a conditional independence claim has observable consequences; writing arrows without checking it is not enough.

The same distinction matters for a diagnostic test. A disease indicator D and a positive result T have a joint law fixed by prevalence, sensitivity, and false-positive rate. Bayes' rule computes $P(D = 1 \mid T = 1)$ from all three; sensitivity alone cannot supply the posterior. For example, prevalence 0.01, sensitivity 0.9, and false-positive rate 0.1 give

$$P(D = 1 \mid T = 1) = \frac{0.9(0.01)}{0.9(0.01) + 0.1(0.99)} = \frac{1}{12} \approx 0.0833.$$

The test result is evidence, yet most positive results in this model are false positives because the condition is rare. Information measures later average log evidence across all possible test outcomes. They do not replace this posterior calculation for an individual result.

3 Entropy and Divergences

Entropy averages the surprisal of outcomes under one distribution, while divergence compares distributions and mutual information measures dependence. This section develops these quantities from log probability, with their assumptions and support conditions made explicit.

3.1 Surprise as Log Probability

For an outcome x with $p_X(x) > 0$, its self-information or surprisal is

$$\imath_X(x) = -\log_2 p_X(x) \quad \text{bits.}$$

An outcome assigned probability $1/2$ has one bit of surprisal; one assigned probability $1/8$ has three bits. The logarithm is useful because independent probabilities multiply while their surprisals add: if the joint probability of two observed outcomes factors as $p_X(x)p_Y(y)$, then $-\log_2 p_{XY}(x, y) = \imath_X(x) + \imath_Y(y)$. This additivity is a design property of the logarithm, not an assertion that arbitrary data sources are independent.

Surprisal is defined relative to a model. The same event can be surprising under one model and ordinary under another. It is also distinct from the Shannon entropy: surprisal is a random quantity evaluated after an outcome, whereas entropy is its expected value before observing that outcome.

For the biased coin with head probability 0.4, a head has surprisal about 1.322 bits and a tail about 0.737 bits. Heads is less probable and therefore has greater surprisal. The entropy averages the two values with probabilities 0.4 and 0.6.

3.2 Shannon Entropy

For a discrete variable X with pmf p_X, Shannon entropy is expected surprisal:

$$H(X) = \mathbb{E}[-\log_2 p_X(X)] = -\sum_{x : p_X(x) > 0} p_X(x) \log_2 p_X(x).$$

Entropy is zero for a deterministic variable. A fair bit has entropy one bit, while a biased coin has less than one bit per toss. If X has at most m possible values,

then $H(X) \leq \log_2 m$, with equality exactly for the uniform law over those m values. The bound is about uncertainty in the distribution, not the number of bits used by a particular file format.

Theorem 3.1 (Entropy bound). If X takes values in a finite alphabet of size m, then $0 \leq H(X) \leq \log_2 m$. The lower equality holds exactly for a point mass; the upper equality holds for the uniform distribution.

Proof sketch. Non-negativity follows because $0 < p(x) \leq 1$ implies $-\log_2 p(x) \geq 0$. For the upper bound, compare p with the uniform distribution $u(x) = 1/m$: non-negativity of $D_{\mathrm{KL}}(p\|u)$ gives $\sum_x p(x) \log_2(mp(x)) = \log_2 m - H(X) \geq 0$. Equality in KL divergence requires $p = u$. □

For the biased coin, $H(X) = -0.4 \log_2(0.4) - 0.6 \log_2(0.6) \approx 0.971$ bits per toss. This expected value is below one even though an individual outcome can have surprisal greater than one bit. There is no contradiction: rare outcomes contribute more, but they occur less often.

The maximum-entropy statement depends on its constraint set. A uniform law is the most uncertain *among distributions on a specified finite alphabet*. There is no normalisable uniform probability density across the whole real line. Among real distributions with a fixed finite variance, a Gaussian maximises differential entropy under the usual regularity assumptions. That fact supports the Gaussian channel and source calculations later, but it does not imply that measurements are actually Gaussian. Maximum entropy describes an extremum under chosen constraints; a physical or empirical model requires evidence beyond the extremum calculation.

Entropy also depends on the granularity of the outcome. If a four-symbol observation is recoded into a binary label that merges three categories, the new label cannot have more entropy than the original observation because it is a deterministic function of it. But the new entropy measures a different variable. Calling the binary label "the entropy of the data" would hide that choice. Conversely, splitting a category according to a new measured attribute expands the random variable and can raise entropy. Always name the source alphabet, sampling interval, and unit when reporting bits per observation.

For a stream with dependence, one-symbol entropy need not be the best compression benchmark. Suppose a fair bit is repeated twice in each pair: $X_{2k} = X_{2k-1}$, and the first bit of each pair is independent across pairs. Every position

has marginal entropy one bit, yet the pair has only one bit of joint entropy. An encoder that knows the pairing can represent the pair with one bit, or half a bit per original symbol, before framing. The chain rule makes the saving explicit: $H(X_{2k-1}, X_{2k}) = H(X_{2k-1}) + H(X_{2k} \mid X_{2k-1}) = 1 + 0$. This is why a memoryless theorem must not be applied to a source with predictable temporal structure by substituting its marginal entropy.

3.2.1 Joint Entropy and Chain Rule

The joint entropy of two finite-alphabet variables is the entropy of their pair:

$$H(X, Y) = -\sum_{x,y} p(x, y) \log_2 p(x, y).$$

Conditional entropy averages the entropy of the conditional distributions:

$$H(X \mid Y) = \sum_{y : p_Y(y) > 0} p_Y(y) H(X \mid Y = y).$$

Theorem 3.2 (Entropy chain rule). For finite-alphabet variables X, Y,

$$H(X, Y) = H(Y) + H(X \mid Y) = H(X) + H(Y \mid X).$$

Proof sketch. For each pair with positive joint probability, factor $p(x, y) = p_Y(y) p(x \mid y)$ and take negative logarithms. Average the sum under $p(x, y)$. The term $-\log p_Y(y)$ averages to $H(Y)$; the term $-\log p(x \mid y)$ averages first conditionally on y and then over y, yielding $H(X \mid Y)$. Reversing the roles gives the other equality. □

For finite alphabets, the sums are finite and the proof is direct. For countable alphabets, the identity holds when the relevant entropies are well-defined and no undefined $\infty - \infty$ subtraction is used. State that integrability qualification if using the identity beyond the finite case.

Repeated application gives $H(X_1, \dots, X_n) = \sum_{i=1}^{n} H(X_i \mid X_1, \dots, X_{i-1})$. If the variables are mutually independent, each conditional term equals its marginal entropy, so joint entropy is the sum. Independence is essential to that simplification.

3.2.2 Conditioning Reduces Entropy

Theorem 3.3 (Conditioning inequality). For finite-alphabet X, Y, $H(X \mid Y) \le H(X)$, with equality if and only if X and Y are independent.

Proof sketch. By the chain rule, $H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) = I(X;Y)$. Section 4 identifies this difference with a KL divergence, which is non-negative and zero exactly when the joint law factors into the product marginals. □

This statement concerns average uncertainty under the joint model. Observing a particular value $Y = y$ can increase a person's uncertainty about X relative to their prior if that observation shifts probability towards a more diffuse conditional law. The theorem says that averaging $H(X \mid Y = y)$ over all y cannot exceed $H(X)$.

3.3 KL Divergence

For distributions P and Q on the same discrete alphabet, the Kullback–Leibler divergence (KL divergence) is

$$D_{\mathrm{KL}}(P\|Q) = \sum_{x:P(x)>0} P(x) \log_2 \frac{P(x)}{Q(x)}.$$

It is finite only if $Q(x) > 0$ wherever $P(x) > 0$ and the sum is finite. If this support condition fails, define the divergence as $+\infty$. The direction matters: $D_{\mathrm{KL}}(P\|Q)$ weights log-ratios using outcomes from P, so swapping P and Q generally changes the answer.

Theorem 3.4 (Gibbs' inequality). For probability distributions P, Q on a finite alphabet, $D_{\mathrm{KL}}(P\|Q) \ge 0$, with equality if and only if $P = Q$; if Q vanishes on the support of P, the divergence is infinite.

Proof sketch. When support inclusion holds, use $\ln t \le t - 1$ for $t > 0$ with $t = Q(x)/P(x)$. Then $P(x)\ln(P(x)/Q(x)) \ge P(x) - Q(x)$. Summing over the support of P yields a non-negative result because Q may also put mass outside that support. Equality requires equality termwise and no extra mass outside, giving $P = Q$. Dividing by $\ln 2$ converts nats to bits. □

KL divergence is not a distance: it can be asymmetric and does not satisfy the

triangle inequality. Its useful interpretation is excess expected log loss under P when probabilities are assessed using Q, as made explicit by cross-entropy below. Do not call it a metric or interpret its numerical value as a geometric distance without additional structure.

The support rule gives KL a sharp failure mode. Let $P = (1/2, 1/2)$ and $Q = (1, 0)$. Because Q declares the second outcome impossible even though P generates it half the time, $D_{\mathrm{KL}}(P\|Q) = +\infty$. In the reverse direction $D_{\mathrm{KL}}(Q\|P) = 1$ bit: data generated under Q always show the first outcome, which P assigns probability one half. The two values are not contradictory; each average is taken under a different generating law. Adding a small positive probability to the second category makes the first divergence finite, but its value depends on that smoothing choice.

Another useful comparison is the Jensen–Shannon divergence. With $M = (P + Q)/2$, define $\mathrm{JSD}(P, Q) = \frac{1}{2}D_{\mathrm{KL}}(P\|M) + \frac{1}{2}D_{\mathrm{KL}}(Q\|M)$. For finite alphabets, the mixture covers both supports, so JSD stays finite even when a directed KL is infinite; it is symmetric and lies between zero and one bit when the mixing weights are equal. It answers a different comparison question and should not replace the directed KL in the cross-entropy identity. In particular, the excess expected log loss incurred by predicting Q on P-generated data remains $D_{\mathrm{KL}}(P\|Q)$, not JSD.

3.3.1 Cross-Entropy and Log Loss

If outcomes follow P but a model assigns probabilities Q, their cross-entropy is

$$H(P, Q) = -\sum_{x} P(x) \log_2 Q(x).$$

Under support inclusion and finite sums, algebra gives

$$H(P, Q) = H(P) + D_{\mathrm{KL}}(P\|Q).$$

If $P(x) > 0$ and $Q(x) = 0$, the cross-entropy and divergence are infinite. The decomposition says that for a fixed true law P, minimising expected log loss over candidate models Q is equivalent to minimising KL divergence from P to Q. For observed samples, average negative log probability is an empirical estimate of

cross-entropy, not necessarily the population quantity.

In classification, if the true label is y and a model reports $q(y)$, the per-example log loss is $-\log q(y)$. For a one-hot target distribution, the cross-entropy reduces to that quantity. With soft targets, the loss averages across target categories. This is a connection to statistical modelling; it does not mean every loss function is entropy or that the model's probabilities are calibrated.

3.4 Entropy of a Bernoulli Variable

For a bit that equals 1 with probability p, entropy is the binary entropy function

$$h_2(p) = -p \log_2 p - (1-p) \log_2 (1-p), \quad 0 \le p \le 1,$$

with endpoint values $h_2(0) = h_2(1) = 0$. It is symmetric around $1/2$, reaches its maximum value of one bit at $p = 1/2$, and approaches zero as the outcome becomes predictable. Differentiating in the interior gives $h_2'(p) = \log_2((1-p)/p)$, so the only stationary point is $p = 1/2$; the second derivative is negative, confirming a maximum.

This simple function recurs in binary channels, error bounds, and rate–distortion examples. It shows that entropy responds to the whole probability balance, not merely the count of possible labels.

3.4.1 Continuous Entropy and Relative Entropy

For a continuous variable with density f_X (when the integral exists), differential entropy is

$$h(X) = -\int f_X(x) \log_2 f_X(x)\, dx.$$

Unlike discrete entropy, differential entropy can be negative and depends on units. If $Y = aX$ for a non-zero scalar a, then $h(Y) = h(X) + \log_2 |a|$. Measuring a length in metres rather than centimetres shifts the numerical differential entropy. Therefore it is unsafe to interpret $h(X)$ by itself as an invariant number of bits needed to encode a continuous value.

Worked example (a uniform measurement). Let X be uniform on an interval

of width $w > 0$, expressed in some physical unit. Its density is $1/w$ on that interval and its differential entropy is $\log_2 w$ in the corresponding coordinate convention. If $w = 1/4$, the value is -2 bits. There is no negative-length binary code hidden in this answer. The value describes a density relative to a coordinate measure; representing a real sample exactly with arbitrary precision still needs unbounded bits in an ordinary digital scheme. If the measurement is rounded to cells of width Δ, its discrete entropy is roughly $h(X) - \log_2 \Delta$ when the density is regular and the cells are fine. The resolution term restores a meaningful discrete coding scale.

Now change only units, measuring the same physical interval in centimetres instead of metres. The numerical width and density change, so $h(X)$ shifts by the logarithm of the conversion factor. A bijective change of units does not alter whether a second variable predicts X; mutual information is invariant under the corresponding change of coordinates. The contrast is why a table of differential entropies across datasets with different measurement units can mislead even when every integral was evaluated correctly.

Relative entropy between continuous distributions instead uses densities with respect to a common reference measure:

$$D_{\mathrm{KL}}(P\|Q) = \int f(x) \log_2 \frac{f(x)}{g(x)}\, dx,$$

when P, Q have densities f, g and the integral is defined. Under suitable absolute continuity and integrability assumptions it is non-negative and invariant under a one-to-one change of coordinates applied to both distributions. If P has mass where Q has none, it is infinite. General distributions need not possess densities; the measure-theoretic definition uses a Radon–Nikodym derivative when P is absolutely continuous with respect to Q, and is infinite otherwise.

For jointly continuous variables, mutual information is safely defined as a KL divergence between the joint law and the product of its marginals. It can also be expressed as $h(X) + h(Y) - h(X, Y)$ when these differential entropies are individually well-defined and the cancellation is legitimate. The KL definition is more robust: differential entropy terms may be infinite even when mutual information is finite. See Continuous Variables and Mutual Information.

3.5 Base R and Python Calculations

For a finite probability vector, define $0 \log 0$ by omitting zero terms. Base R's `log2()` gives results in bits. Logical indexing selects the positive probabilities before logarithms are evaluated.

R

```
entropy <- function(p) {
  stopifnot(is.numeric(p), length(p) > 0L, all(is.finite(p)),
            all(p >= 0), abs(sum(p) - 1) < 1e-12)
  positive <- p > 0
  -sum(p[positive] * log2(p[positive]))
}

entropy(c(0.4, 0.6))
entropy(c(0.5, 0.5))
entropy(c(1, 0))
```

Python

```
from math import isfinite, log2


def entropy(probabilities):
    probabilities = list(probabilities)
    if not probabilities or any(not isfinite(p) or p < 0 for p in probabilities):
        raise ValueError("probabilities must be finite and non-negative")
    if not abs(sum(probabilities) - 1.0) < 1e-12:
        raise ValueError("probabilities must sum to one")
    return -sum(p * log2(p) for p in probabilities if p > 0)


print(entropy([0.4, 0.6]))
print(entropy([0.5, 0.5]))
print(entropy([1, 0]))
```

The results are approximately 0.971, 1, and 0 bits. The validation is part of the example: normalising arbitrary non-negative weights silently changes the supplied model, so code should either require a probability vector or make normalisation an explicit operation.

To compare two distributions, handle support violations before evaluating logarithms. Supply both vectors in the same category order: equal lengths alone do not establish that corresponding entries represent the same outcome.

R

```
kl_divergence <- function(p, q) {
  stopifnot(is.numeric(p), is.numeric(q), length(p) > 0L,
            length(p) == length(q), all(is.finite(p)), all(is.finite(q)),
            all(p >= 0), all(q >= 0),
            abs(sum(p) - 1) < 1e-12, abs(sum(q) - 1) < 1e-12)
  if (any(p > 0 & q == 0)) return(Inf)
  keep <- p > 0
  sum(p[keep] * log2(p[keep] / q[keep]))
}
```

```r
kl_divergence(c(0.4, 0.6), c(0.5, 0.5))
kl_divergence(c(0.5, 0.5), c(1, 0))
kl_divergence(c(1, 0), c(0.5, 0.5))
```

Python

```python
from math import isfinite, log2


def kl_divergence(p, q):
    p, q = list(p), list(q)
    if not p or len(p) != len(q):
        raise ValueError("probability vectors must have equal, non-zero lengths")
    for probabilities in (p, q):
        if any(not isfinite(v) or v < 0 for v in probabilities):
            raise ValueError("probabilities must be finite and non-negative")
        if not abs(sum(probabilities) - 1.0) < 1e-12:
            raise ValueError("probabilities must sum to one")
    if any(pi > 0 and qi == 0 for pi, qi in zip(p, q)):
        return float("inf")
    return sum(pi * log2(pi / qi) for pi, qi in zip(p, q) if pi > 0)


print(kl_divergence([0.4, 0.6], [0.5, 0.5]))
print(kl_divergence([0.5, 0.5], [1, 0]))
print(kl_divergence([1, 0], [0.5, 0.5]))
```

The first result is approximately 0.029 bits; the last two are `Inf` and 1 bit, reproducing the support and direction example above.

These functions are suitable for small examples. For large or extreme probability vectors, compute with stable log-probabilities and avoid underflow; do not interpret a tiny negative floating-point result as evidence that KL divergence is mathematically negative.

4 Information Between Variables

Entropy and divergence lead to mutual information by comparing a joint law with the law that would result if its marginals were independent. This construction makes the quantity symmetric in its two arguments, although the underlying conditional uncertainties can be very different. A small mutual information can mean weak dependence, or it can reflect a target with very little entropy to begin with. State the marginal entropies alongside it when interpreting a result.

Worked example (four-symbol mismatch). For the recurring source $p = (0.4, 0.3, 0.2, 0.1)$, its entropy is about 1.846 bits. Suppose a probability model uses $q = (0.25, 0.25, 0.25, 0.25)$. Its ideal log-loss code spends exactly two bits on every outcome, so the expected penalty relative to the true source law is

$2 - H(p) \approx 0.154$ bits per symbol. The reversed divergence $D_{\mathrm{KL}}(q\|p)$ answers a different question: expected model mismatch if outcomes were uniform and predictions used p. It is about 0.176 bits. Neither direction by itself includes the integer codeword and header costs of a real file format.

This example also separates surprise from realised code length. Symbol d has true surprisal $-\log_2 0.1 \approx 3.322$ bits and gets three bits in the Huffman code below. The single codeword is shorter than its surprisal; the average code length remains above entropy. A claim that every symbol must cost at least its own surprisal would contradict this ordinary prefix code. The Kraft constraint acts on the whole code tree, so code lengths must be judged together.

4.1 Conditional Entropy as Remaining Uncertainty

Conditional entropy $H(X \mid Y)$ is the expected uncertainty about X after observing Y. It averages the entropy of the posterior distribution $p(x \mid y)$ over y. It does not mean the entropy of one particular posterior, and an application should state whether it is discussing a realised observation or the model-wide average.

The chain rule from Joint Entropy and Chain Rule gives $H(X, Y) = H(Y) + H(X \mid Y)$. This identity holds for finite alphabets without an independence assumption. In fact, it is the definition of conditional entropy consistent with the joint entropy decomposition. For countable variables, use the same finiteness and well-definedness cautions stated there.

For the noisy-bit example with a fair X and independent flip probability 0.1, the posterior after either value of Y has probabilities $(0.9, 0.1)$. Therefore $H(X \mid Y) = h_2(0.1) \approx 0.469$ bits. Since $H(X) = 1$, observing Y removes about 0.531 bits of uncertainty on average.

4.2 Mutual Information

Mutual information quantifies statistical dependence between two variables. For finite alphabets, equivalent expressions are

$$
\begin{aligned}
I(X;Y) &= H(X) - H(X \mid Y) \\
&= H(Y) - H(Y \mid X) \\
&= H(X) + H(Y) - H(X,Y) \\
&= D_{\mathrm{KL}}(P_{XY} \| P_X P_Y) \\
&= \sum_{x,y : p(x,y)>0} p(x,y) \log_2 \frac{p(x,y)}{p_X(x) p_Y(y)}.
\end{aligned}
$$

The expression as a KL divergence proves non-negativity and identifies the zero case. $I(X;Y) = 0$ exactly when X and Y are independent. Symmetry follows from the joint-law expression or the entropy identity: $I(X;Y) = I(Y;X)$. Also $I(X;Y) \le \min(H(X), H(Y))$ for finite entropies, because conditional entropy is non-negative.

Worked Example 4.1 (A binary symmetric observation). Let X be a fair bit and let Y be X flipped with probability 0.1, independently of X. The output Y is fair, so $H(Y) = 1$. Given X, uncertainty about Y is $h_2(0.1) \approx 0.469$. Thus $I(X;Y) = H(Y) - H(Y \mid X) = 1 - h_2(0.1) \approx 0.531$ bits. The observation is informative but not sufficient to recover X without error.

We can verify the same answer without starting from the entropy difference. The four joint probabilities for $(X,Y) = (0,0), (0,1), (1,0), (1,1)$ are $(0.45, 0.05, 0.05, 0.45)$, while every product of marginals is 0.25. The two matching outcomes each contribute $0.45 \log_2(0.45/0.25)$ to the expected log ratio; the two mismatches each contribute $0.05 \log_2(0.05/0.25)$. A mismatch contributes a negative *pointwise* value because it is less likely under the joint law than under independence. Their weighted sum is non-negative. An individual information-density term may therefore be negative even though mutual information cannot be.

The fair input matters. If $P(X = 1) = a$, the output probability becomes $P(Y = 1) = 0.1 + 0.8a$ and $H(Y \mid X)$ remains $h_2(0.1)$. Hence $I_a(X;Y) = h_2(0.1 + 0.8a) - h_2(0.1)$. For $a = 0$ the input never varies, and mutual information is zero despite the same physical channel. At $a = 1/2$ the output is fair and the value is 0.531 bits. The channel chapter maximises over a to obtain capacity; the mutual information just calculated belongs to one specified input law. That distinction holds for every channel, even when symmetry happens to make the fair law optimal.

Mutual information is not a measure of semantic relevance by itself. A variable can carry information about a target in the probabilistic sense but be inaccessible to a particular algorithm, too costly to use, or ethically inappropriate to use. Those are separate modelling and decision questions.

4.2.1 Conditional Mutual Information and Chain Rules

Conditional mutual information is the average dependence between X and Y within the conditional distributions given Z:

$$I(X;Y \mid Z) = \sum_z p(z) D_{\mathrm{KL}}\Big(P_{XY|Z=z} \Big\| P_{X|Z=z} P_{Y|Z=z}\Big) .$$

For finite alphabets it is non-negative, and it is zero exactly when X and Y are conditionally independent given Z, up to conditioning values of probability zero. Entropy identities include

$$I(X;Y \mid Z) = H(X \mid Z) - H(X \mid Y,Z) = H(Y \mid Z) - H(Y \mid X,Z).$$

The mutual-information chain rule is

$$I(X;Y,Z) = I(X;Y) + I(X;Z \mid Y) = I(X;Z) + I(X;Y \mid Z).$$

These formulas follow from the entropy chain rule and do not require X, Y, Z to be independent. They do require the entropies or divergences to be well-defined; finite alphabets suffice. The non-negativity of conditional mutual information is the key reason conditioning can be used in the data-processing proof.

Conditioning can increase mutual information: $I(X;Y \mid Z)$ is not generally smaller than $I(X;Y)$. This differs from the inequality for conditional entropy and is a frequent source of mistaken intuition. For example, conditioning on a common effect can induce dependence between otherwise independent causes (a collider pattern). In finite-alphabet information identities, no monotonicity claim about conditional mutual information should be added without a specific hypothesis.

The parity example makes the increase exact. Let X and Y be independent fair bits, and let $Z = X \oplus Y$. Before seeing Z, the inputs are independent, so

$I(X;Y) = 0$. Once Z is fixed, knowing X determines Y, while X remains fair. Thus $H(Y \mid Z) = 1$ and $H(Y \mid X, Z) = 0$, giving $I(X;Y \mid Z) = 1$ bit. Conditioning on a variable constructed from both inputs has turned an independent pair into a perfectly dependent conditional pair. Nothing about this calculation implies that observing Z caused either input.

There is an equally simple case in the other direction. If $X = Y$ is a fair bit and $Z = X$, then $I(X;Y) = 1$ bit but $I(X;Y \mid Z) = 0$: once the shared bit is given, there is no uncertainty left to reduce. These two examples show why conditional mutual information must be calculated from the conditional law rather than inferred from a generic slogan about conditioning.

4.3 Independence, Synergy, and Interaction

Pairwise mutual information can miss higher-order structure. Let X, Y be independent fair bits and $Z = X \oplus Y$. Then each pair is independent, so each pairwise mutual information is zero, yet any two variables determine the third. The joint entropy is two bits, not three, and $H(Z \mid X, Y) = 0$. This parity example shows why "no pairwise dependence" does not imply joint independence.

Information shared by two observations can be redundant, complementary, or mixed. The basic definitions do not supply a unique decomposition into these categories. Terms such as synergy require a chosen framework and should not be introduced as if they were uniquely determined by Shannon entropy. For this primer, use the parity example to expose the limitation and route specialised decompositions to further reading.

4.4 Data-Processing Inequality

Theorem 4.1 (Data-processing inequality). Let X, Y, Z be finite-alphabet random variables forming the Markov chain $X \to Y \to Z$, meaning X and Z are conditionally independent given Y. Then

$$I(X;Z) \leq I(X;Y).$$

Proof sketch. By the mutual-information chain rule, $I(X;Y,Z) = I(X;Y) + I(X;Z \mid Y)$. The Markov condition makes the conditional term zero, so $I(X;Y,Z) = I(X;Y)$. Expanding in the other order gives $I(X;Y,Z) =$

$I(X;Z) + I(X;Y \mid Z)$. Conditional mutual information is non-negative, hence the inequality. For general measurable variables, the result holds under the measure-theoretic definition of mutual information and appropriate regular conditional distributions; the finite-alphabet proof avoids those technicalities. □

Equality holds precisely when $I(X;Y \mid Z) = 0$ under this proof, equivalently when X and Y are conditionally independent given Z (for finite alphabets). A deterministic map $Z = f(Y)$ can preserve all information about X if it retains a sufficient statistic for X relative to Y; determinism alone does not guarantee equality.

The Markov hypothesis is essential. If Z receives information directly from X as well as from Y, then the arrows do not form $X \to Y \to Z$ and the inequality need not apply. The theorem says that a processing stage cannot increase information about X when its output is generated from Y alone according to a fixed conditional law. It does not say processing cannot make a downstream task easier for a limited model or a human reader.

In the two-stage flip example, the Markov property holds when the flips are independent as specified. A fair X gives $I(X;Y) = 1 - h_2(a)$ and $I(X;Z) = 1 - h_2(a+b-2ab)$. For $a = 0.1$ and $b = 0.2$, these are about 0.531 and 0.173 bits. The second observation carries less information about the original bit. One need not prove this by differentiating binary entropy: the data-processing theorem already gives the ordering once the conditional factorisation is checked. If $b = 1$, the second stage deterministically inverts Y and equality holds; processing has changed the label but lost no information.

A statistical transformation can still improve performance for a restricted learner. For example, a classifier that only accepts one scalar feature might perform better on a carefully designed summary $Z = f(Y)$ than on a raw high-dimensional Y it cannot fit with available data. The population mutual information cannot increase under this mapping, but estimation error, computation, and inductive bias can change. Separate the theorem's distributional statement from an empirical model comparison.

Example. Suppose Y is a noisy measurement of a sensitive variable X, and Z is a report generated only from Y. Then $I(X;Z) \leq I(X;Y)$. This is a statement about average statistical dependence under a specified joint law. It does

not establish a formal privacy guarantee by itself: adversary knowledge, rare individuals, utility, and the chosen threat model still matter.

4.5 Continuous Variables and Mutual Information

For arbitrary random variables on suitable measurable spaces, define mutual information by relative entropy:

$$I(X;Y) = D_{\mathrm{KL}}(P_{XY} \| P_X \times P_Y)\,.$$

This definition extends the discrete one and remains meaningful when differential entropies are not. It can be infinite. If the joint law is not absolutely continuous with respect to the product of its marginals, mutual information is infinite; deterministic relationships between continuous variables often produce this phenomenon. For example, if $Y = X$ and X has a continuous density, the joint law lies on the diagonal in the plane while the product law assigns that diagonal probability zero, so $I(X;Y) = +\infty$.

When relevant differential entropies are finite and subtraction is valid, one may write $I(X;Y) = h(X) - h(X \mid Y) = h(X) + h(Y) - h(X,Y)$. Do not use this identity when it creates an undefined $\infty - \infty$ expression. Mutual information, unlike differential entropy, is invariant under separate invertible coordinate transformations of X and Y and is non-negative.

For jointly Gaussian scalar variables with non-degenerate variances and correlation ρ satisfying $|\rho| < 1$, the mutual information is $-\frac{1}{2}\log_2(1-\rho^2)$ bits. At $|\rho| = 1$, one variable is an exact affine function of the other and the mutual information diverges. This example is useful but not a general formula for arbitrary continuous distributions; correlation alone does not determine mutual information outside specified families such as the jointly Gaussian case.

At $\rho = 0.6$, the jointly Gaussian formula gives $-\frac{1}{2}\log_2(0.64) \approx 0.322$ bits. Because a jointly Gaussian pair with zero correlation is independent, the formula gives zero at $\rho = 0$. Neither conclusion extends to an arbitrary joint law. For example, let a symmetric continuous X have finite variance and set $Y = X^2$. The covariance of X and Y can be zero by symmetry while Y is completely determined by X; in the ideal noise-free continuous model the MI is generally infinite. Adding measurement noise changes that value. State both the distributional

family and measurement resolution before using correlation as an information proxy.

4.6 Base R and Python Calculation From a Joint Table

For a small finite joint distribution, mutual information can be computed from the ratio of joint mass to the product of marginals. This form makes the independence comparison explicit. Matrix margins provide the marginals and logical indexing selects positive cells. `outer()` constructs the product-marginal matrix with the same dimensions as the joint table.

R

```r
joint <- matrix(c(0.4, 0.1,
                  0.1, 0.4),
                nrow = 2, byrow = TRUE)
stopifnot(all(is.finite(joint)), all(joint >= 0),
          abs(sum(joint) - 1) < 1e-12)
px <- rowSums(joint)
py <- colSums(joint)
independent <- outer(px, py)
positive <- joint > 0

information <- sum(joint[positive] *
                   log2(joint[positive] / independent[positive]))
information
```

Python

```python
from math import isfinite, log2

joint = [[0.4, 0.1],
         [0.1, 0.4]]
assert joint and joint[0]
assert all(len(row) == len(joint[0]) for row in joint)
assert all(isfinite(p) and p >= 0 for row in joint for p in row)
assert abs(sum(sum(row) for row in joint) - 1.0) < 1e-12
px = [sum(row) for row in joint]
py = [sum(column) for column in zip(*joint)]

information = sum(
    p * log2(p / (px[i] * py[j]))
    for i, row in enumerate(joint) for j, p in enumerate(row)
    if p > 0
)
print(information)
```

The result is approximately 0.278 bits. This is population mutual information for the specified table, not an estimate from sampled observations. The validation requires a normalised joint distribution. For data frames of observations, first make the state space and handling of unobserved category combinations explicit; `table()` can construct empirical counts, but naive plug-in mutual in-

formation is biased upward in many finite-sample settings. See Estimating Information from Data.

5 Lossless Source Coding

Lossless source coding turns probabilities into binary representations and evaluates their cost by expected code length. The section relates prefix-code constraints to entropy bounds, then develops practical constructions and the conditions behind block-coding results.

5.1 From Probabilities to Code Lengths

A binary code maps source symbols to finite strings over $\{0, 1\}$. If symbol x receives codeword $c(x)$, its length is $\ell(x) = |c(x)|$. Under source distribution p, the expected length is

$$L = \mathbb{E}[\ell(X)] = \sum_x p(x)\ell(x)$$

bits per source symbol. A code is injective if distinct symbols have distinct codewords. Injectivity permits decoding one isolated codeword, but concatenated variable-length codewords need a parsing rule.

A prefix code has no codeword that is a prefix of another. It can therefore be decoded instantaneously from a concatenated bitstream without separators or lookahead beyond the end of a codeword. Prefix-freeness is sufficient for unique decodability, though not necessary: some uniquely decodable codes are not prefix codes. The simplest entropy bounds are stated for prefix codes; do not silently claim that every argument about a prefix tree applies to arbitrary codes without a corresponding theorem.

5.1.1 Kraft Inequality

For any binary prefix code with lengths $\ell_1, \dots, \ell_m$,

$$\sum_{i=1}^{m} 2^{-\ell_i} \leq 1.$$

Each codeword identifies a node in a full binary tree. A codeword of length ℓ_i has a descendant region occupying a fraction $2^{-\ell_i}$ of all paths of a sufficiently deep fixed length. Prefix-freeness makes these regions disjoint, so their fractions sum to at most one.

Theorem 5.1 (Kraft's inequality and its converse). The lengths of any binary prefix code satisfy Kraft's inequality. Conversely, any finite list of non-negative integer lengths satisfying the inequality can be realised by a binary prefix code.

Proof sketch. Necessity is the disjoint-tree-region argument above. For sufficiency, sort the lengths in non-decreasing order and assign leaves greedily from left to right in the binary tree. At depth ℓ, each selected leaf consumes $2^{-\ell}$ of the unit interval of binary paths. The Kraft sum ensures total consumed capacity is at most one, so no assigned leaf needs to overlap an earlier leaf. Equivalently, the canonical-code construction assigns each codeword the next available binary interval of width $2^{-\ell}$. □

The converse concerns existence of a prefix code with those lengths, not a particular bit assignment. Many bit patterns can have the same length profile and expected cost.

5.1.2 The Entropy Lower Bound

Theorem 5.2 (Expected length lower bound). For a discrete source X with finite alphabet and distribution p, every binary prefix code has expected length $L \geq H(X)$.

Proof sketch. Let $K = \sum_x 2^{-\ell(x)} \leq 1$ and define a normalised distribution $q(x) = 2^{-\ell(x)}/K$. Then

$$D_{\mathrm{KL}}(p\|q) = -H(X) + \sum_x p(x)\ell(x) + \log_2 K \geq 0.$$

Since $\log_2 K \leq 0$, rearranging yields $L \geq H(X) - \log_2 K \geq H(X)$. Symbols with zero source probability can be omitted or assigned arbitrary unused codewords. □

This theorem applies to expected length under the stated source law and a prefix code. It does not say that each individual codeword must have length at least that symbol's surprisal, nor that a finite block always uses exactly $nH(X)$ bits.

An individual common symbol may have a short codeword and a rare one a longer codeword; the inequality holds after averaging.

The entropy lower bound also extends to uniquely decodable codes through the Kraft–McMillan inequality: their length sequences obey the same Kraft bound. Prefix codes provide the construction and parsing intuition here. An arbitrary injective variable-length code is insufficient, because concatenated codewords may be ambiguous.

5.2 Shannon Code and the One-Bit Redundancy Bound

For each positive-probability symbol, choose length $\ell(x) = \lceil -\log_2 p(x) \rceil$. Then $2^{-\ell(x)} \leq p(x)$, so Kraft's inequality holds and a prefix code exists. Since $\lceil a \rceil < a + 1$ unless a is an integer (where equality with a holds), its expected length satisfies

$$H(X) \leq L < H(X) + 1$$

for a finite source after an appropriate prefix code is assigned. The lower bound is Theorem 5.2; the upper bound follows by averaging $\ell(x) < -\log_2 p(x) + 1$.

The code is often called a Shannon code, but the name can refer to closely related constructions in different texts. The robust claim is the length assignment and its bound. A code word length is an integer, while surprisal is generally fractional; the ceiling is the rounding loss. This construction need not be optimal among one-symbol prefix codes.

5.3 Huffman Coding and a Complete Example

Huffman coding produces a minimum-expected-length binary prefix code for a known finite source when symbols are encoded individually and symbol costs are measured by codeword length. Repeatedly merge the two least probable items, treating a merged item as a subtree; assign 0 and 1 to the branches after the tree is complete.

Consider probabilities $p(a) = 0.4$, $p(b) = 0.3$, $p(c) = 0.2$, and $p(d) = 0.1$. Merge d and c to form a node of mass 0.3; merge that node with b to form 0.6; then merge with a. The table gives one valid assignment. Ties can produce different trees and codewords without changing the optimal expected length.

Symbol	Probability	Codeword	Length
a	0.40	`0`	1
b	0.30	`10`	2
c	0.20	`110`	3
d	0.10	`111`	3

The code is prefix-free, and its expected length is

$$L = 0.4(1) + 0.3(2) + 0.2(3) + 0.1(3) = 1.9 \text{ bits per symbol.}$$

The source entropy is $H(X) \approx 1.846$ bits, so the redundancy is about 0.054 bits per symbol. The code achieves the minimum expected length among binary prefix codes for this one-symbol problem. It is not exact entropy coding in the sense of assigning each symbol precisely $-\log_2 p(x)$ bits, because lengths are integers.

The result changes if the source probabilities change while the codewords stay fixed. Suppose the actual law becomes $r = (0.1, 0.2, 0.3, 0.4)$ in the same symbol order. The displayed tree then averages $0.1(1) + 0.2(2) + 0.3(3) + 0.4(3) = 2.6$ bits per symbol, even though the entropy is still about 1.846 bits: permuting the probabilities leaves entropy unchanged but makes this fixed code badly matched to the new frequencies. Rebuilding the Huffman tree for r restores an expected length of 1.9 bits. A code optimal for one distribution need not be optimal for another.

The mismatch cost for a fixed integer code is not exactly a KL divergence. KL describes ideal lengths $-\log_2 q(x)$ under a model q; a prefix code rounds and couples lengths through Kraft's inequality. If a compressor sends the new model to its decoder, those model bits must also be counted. For a long stream the model cost can be amortised, but on a short message it may dominate the saved fraction of a bit per symbol. This is why an entropy figure alone cannot predict a compressed file size.

Proof sketch of Huffman optimality. In an optimal binary prefix tree, the two least probable symbols can be chosen as sibling leaves at maximum depth: if not, swapping a lower-probability symbol deeper cannot increase cost. Merge those two leaves into a single symbol whose probability is their sum. Any code for the

reduced source expands to a code for the original source by appending one bit to each of the two merged symbols; the expected length increases by exactly their combined probability. Induction on alphabet size proves that choosing the two least probable symbols at each step is optimal. This argument assumes symbol-wise coding, binary branches, and known probabilities; it says nothing about finite-block global optimality or model cost.

5.3.1 Base R and Python for Expected Length

For a known codebook, expected length is a weighted sum. Named vectors keep probabilities and codewords aligned by symbol. The checks below require matching, unique symbol names, a valid probability vector, and distinct binary codewords. They also verify prefix-freeness: for each codeword, `startsWith()` checks whether any other codeword begins with it. This computes the cost of the supplied code; it does not construct a Huffman tree.

R

```
probability <- c(a = 0.4, b = 0.3, c = 0.2, d = 0.1)
code <- c(a = "0", b = "10", c = "110", d = "111")
stopifnot(!is.null(names(probability)), !is.null(names(code)),
          !anyDuplicated(names(probability)), !anyDuplicated(names(code)),
          setequal(names(probability), names(code)),
          all(is.finite(probability)), all(probability >= 0),
          abs(sum(probability) - 1) < 1e-12)
code <- code[names(probability)]
stopifnot(!anyNA(code), !anyDuplicated(code), grepl("^[01]+$", code))
has_prefix <- vapply(seq_along(code), function(i) {
  any(startsWith(code[-i], code[i]))
}, logical(1))
stopifnot(!any(has_prefix))
expected_length <- sum(probability * nchar(code))
expected_length
```

Python

```
from math import isfinite

probability = {"a": 0.4, "b": 0.3, "c": 0.2, "d": 0.1}
code = {"a": "0", "b": "10", "c": "110", "d": "111"}

assert probability.keys() == code.keys()
assert all(isfinite(p) and p >= 0 for p in probability.values())
assert abs(sum(probability.values()) - 1.0) < 1e-12
words = list(code.values())
assert all(isinstance(word, str) and word and set(word) <= {"0", "1"}
           for word in words)
assert len(set(words)) == len(words)
assert all(not right.startswith(left)
           for i, left in enumerate(words) for j, right in enumerate(words)
           if i != j)
expected_length = sum(probability[symbol] * len(code[symbol])
```

```
                      for symbol in probability)
print(expected_length)
```

The result is 1.9 bits per symbol, matching the worked calculation. Avoid presenting a short hand-written tree builder as production-quality Huffman code. Tie handling, zero-probability symbols, arbitrary alphabets, and canonical-code serialisation deserve deliberate design. The example checks the supplied non-empty codewords and expected-length arithmetic; a one-symbol alphabet can instead use an empty codeword when framing supplies the symbol count.

5.4 Block Coding and the Noiseless Source-Coding Theorem

Single-symbol codes pay an integer-length penalty on each symbol. A block code groups n source symbols into one super-symbol, so the code length per original symbol can approach entropy as n grows. For an i.i.d. finite-alphabet source, the block distribution is $p(x_1, \dots, x_n) = \prod_{i=1}^{n} p(x_i)$ and its entropy is $nH(X)$.

Theorem 5.3 (Noiseless source-coding theorem, finite-alphabet i.i.d. form). Let $X_1, X_2, \dots$ be independent and identically distributed (i.i.d.) with a finite alphabet and entropy $H(X)$. For every $\varepsilon > 0$, sufficiently large block lengths admit exactly lossless prefix codes for $X^n = (X_1, \dots, X_n)$ whose expected rate is less than $H(X) + \varepsilon$ bits per source symbol. Conversely, every exactly lossless uniquely decodable block code has expected rate at least $H(X)$ for each block. A fixed-length formulation that permits a vanishing block-error probability has a related threshold theorem, but its rate and error convention differ from this expected-length statement.

Proof sketch. For achievability, assign Shannon lengths to the n-symbol super-alphabet. The expected block length is below $H(X^n) + 1 = nH(X) + 1$, hence per-symbol rate is below $H(X) + 1/n$. For the converse in an exactly lossless prefix setting, apply the entropy lower bound to the block: expected length is at least $H(X^n) = nH(X)$. The Kraft–McMillan extension gives the same expected-length lower bound for a uniquely decodable block code. If decoding errors are permitted, a separate fixed-length converse uses typical sets or an appropriate entropy bound; its exact statement depends on the coding formulation. □

This elementary expected-length statement does not require an informal typical-set proof. The typical-set argument gives useful intuition: under i.i.d. finite-alphabet sampling, most probability lies on sequences whose probability is near $2^{-nH(X)}$, so their count is roughly $2^{nH(X)}$. A lossless code must distinguish the

high-probability sequences, suggesting a rate near $H(X)$. To make the converse rigorous, handle atypical sequences and the error criterion rather than treating "roughly" as a theorem.

For the four-symbol source, the one-symbol Huffman code costs 1.9 bits on average. If we use Shannon's length assignment for every n-symbol block, its average block cost is below $n(1.8464) + 1$ bits, or below $1.8464 + 1/n$ per source symbol. At $n = 100$, this upper bound is approximately 1.8564 bits per symbol. It is an existence bound for a large block alphabet, not a recommendation to build a code table with 4^{100} entries. Arithmetic or range coding approaches the same fractional-length idea with a compact evolving interval and model.

There are two different meanings of "rate" in nearby coding statements. Here rate is *expected bits emitted per source symbol* by an exactly invertible variable-length encoder. A fixed-length source block code instead emits a predetermined number of bits for every block and may declare an error on atypical source sequences. For such a code, a rate above entropy can make error probability vanish as block length grows; below entropy, vanishing error is impossible under the standard i.i.d. finite-alphabet formulation. An expected-length code has no block error, but its realised length varies across source blocks. Mixing these rate conventions can turn a correct theorem into a false operational claim.

For a stationary source with memory, the relevant asymptotic rate is generally the entropy rate $\lim_{n\to\infty} H(X^n)/n$ when this limit exists, not necessarily the one-symbol entropy $H(X_1)$. A stationary ergodic finite-alphabet source has an entropy rate and a corresponding asymptotic equipartition property. Do not call a dependent source memoryless or substitute $H(X_1)$ for its entropy rate.

5.4.1 What "lossless" Means in Practice

Lossless coding requires exact recovery of the encoded source sequence under the declared code and decoding rule. A compression algorithm may be lossless relative to a byte sequence even if an earlier data-cleaning step discarded information; the boundary of what counts as the source must be specified.

The entropy theorem assumes the encoder and decoder share or communicate enough about the coding model. If a compressor uses probabilities estimated from data, the model description has a cost unless it is fixed in advance or otherwise shared. Under a mismatched model q, ideal log-loss length averages to

cross-entropy $H(p, q) = H(p) + D_{\mathrm{KL}}(p \| q)$, before finite-precision and framing overhead. This links coding back to Section 3.6.

Finite files also pay for headers, dictionaries, block boundaries, checksums, and escape symbols. A code that is asymptotically close to entropy can expand a short input after those costs. A model that assigns zero probability to a possible future symbol can make an ideal arithmetic code impossible; practical coders use smoothing or escape mechanisms. These implementation details qualify practical compression claims but do not alter the entropy lower bound under its mathematical assumptions.

A practical audit starts by fixing what exactly must be recovered. If a sensor stores a rounded decimal value, a lossless byte compressor can reproduce that rounded byte sequence perfectly while the original analogue signal is already lost. If an encoder normalises Unicode text before compression, the exact original byte sequence may likewise be unrecoverable. Such preprocessing must be accounted for outside the lossless coding theorem. Similarly, a checksum can detect a damaged compressed stream but does not generally repair it; protection against channel errors belongs to the channel-code model.

5.5 Arithmetic Coding: Probability Intervals

Arithmetic coding illustrates how variable-length bitstrings can approximate fractional surprisal across a sequence. Given a known finite probability model, begin with interval $[0, 1)$. Divide it into subintervals whose widths match symbol probabilities. After each symbol, retain the subinterval for that symbol and rescale within it. After a sequence $x_1, \ldots, x_n$, the interval width is $\prod_i p(x_i)$ for an i.i.d. model, so the number of bits needed to identify a point in it is approximately

$$-\log_2 \prod_{i=1}^{n} p(x_i) = \sum_{i=1}^{n} -\log_2 p(x_i).$$

For $p(A) = 0.75$ and $p(B) = 0.25$, the sequence `AB` maps to an interval of width $0.75(0.25) = 0.1875$, whose ideal information is about 2.415 bits. No individual two-bit string has fractional length; the interval method assigns precision across the entire sequence and finalisation adds bounded overhead under an actual finite-precision implementation.

This is an intuition, not a complete arithmetic coder. A real implementation

must manage finite registers, renormalisation, underflow, termination, and a probability model known to both sides. The next chapter contrasts this source representation with channel coding, where redundancy is intentionally added to enable recovery from noise.

6 Noisy Channels and Capacity

Compression and communication are related but answer different questions. Source coding asks how concisely a source can be represented. Channel coding asks how reliably a message can be recovered after a specified channel has altered the transmitted symbols. Entropy and mutual information connect the questions, but source entropy is not a channel's capacity, and a code's rate is not automatically its useful throughput.

This section introduces a probabilistic channel model, computes mutual information for selected input laws, optimises that law to obtain capacity, and states the asymptotic coding theorem. It treats a discrete memoryless channel first, then compares the binary symmetric channel (BSC) with the binary erasure channel (BEC), considers an asymmetric channel, and closes with the real additive white Gaussian noise (AWGN) model. The theorem statements concern mathematical existence; they do not supply a practical encoder, decoder, latency target, or finite-block error guarantee.

6.1 A Channel as a Conditional Distribution

A channel is a conditional probability distribution from an input alphabet $\mathcal{X}$ to an output alphabet $\mathcal{Y}$. Write its transition law as $W(y \mid x) = P(Y = y \mid X = x)$. For each input x, $W(\cdot \mid x)$ is a distribution over possible outputs. The channel describes what can happen to a transmitted symbol; an input distribution $p(x)$ describes how often the transmitter chooses each input. These are separate ingredients.

A discrete memoryless channel (DMC) has finite input and output alphabets and the same transition law at each use. Conditional on the entire input sequence, outputs are independent:

$$P(y_1, \dots, y_n \mid x_1, \dots, x_n) = \prod_{i=1}^{n} W(y_i \mid x_i).$$

The memoryless condition concerns the channel noise, not the encoder's choices. The inputs may be dependent because a codeword is selected as a whole. Nor does memorylessness mean that a physical system has no state: it means the chosen model either has no relevant state or has already incorporated a state in a way that makes this conditional factorisation valid. A radio link with slow fading or a sensor with burst errors may violate it.

Given input law $p(x)$, the joint distribution is $p(x, y) = p(x)W(y \mid x)$, and the output marginal is $p_Y(y) = \sum_x p(x)W(y \mid x)$. The mutual information for this input law is

$$I_p(X;Y) = \sum_{x,y} p(x)W(y \mid x) \log_2 \frac{W(y \mid x)}{p_Y(y)} = H(X) - H(X \mid Y).$$

The subscript reminds us that the value depends on the distribution used to select X, as well as on the channel. The conditional entropy asks how uncertain the input remains after observing the output. This connects directly to Mutual Information: mutual information is a property of a joint distribution, not of a channel diagram alone.

6.2 Binary Symmetric Channel: One Input Law, One Capacity

A BSC has input and output alphabets $\{0, 1\}$. Each input bit is flipped independently with crossover probability ε, where $0 \le \varepsilon \le 1$. Thus $W(1 \mid 0) = W(0 \mid 1) = \varepsilon$ and $W(0 \mid 0) = W(1 \mid 1) = 1 - \varepsilon$. Let $X \sim \text{Bernoulli}(a)$, so $P(X = 1) = a$, and let $Z \sim \text{Bernoulli}(\varepsilon)$ be independent noise. The output is $Y = X \oplus Z$, with $P(Y = 1) = \varepsilon + a(1 - 2\varepsilon)$.

The noise bit is independent of X, and Y given X is just a translated version of that noise. Therefore $H(Y \mid X) = h_2(\varepsilon)$, where $h_2(u) = -u \log_2 u - (1 - u) \log_2(1 - u)$ and $0 \log_2 0 = 0$. It follows that

$$I_a(X;Y) = h_2(\varepsilon + a(1 - 2\varepsilon)) - h_2(\varepsilon).$$

This formula separates the *achieved mutual information* for input probability a from the *channel capacity*, which maximises that quantity over all allowed input laws. At $a = 1/2$, the output is uniform and $I_{1/2}(X;Y) = 1 - h_2(\varepsilon)$. For the

BSC, this uniform input is capacity-achieving: $H(Y) \leq 1$, with equality for a uniform binary output, while $H(Y \mid X) = h_2(\varepsilon)$ for every input law. Hence

$$C = \max_{0 \leq a \leq 1} I_a(X;Y) = 1 - h_2(\varepsilon) \quad \text{bits per channel use.}$$

The symmetry argument applies also at $\varepsilon = 0$ and $\varepsilon = 1$. At the latter endpoint, the channel deterministically flips every bit; it still has capacity one bit per use because the receiver can invert the flip. At $\varepsilon = 1/2$, the output is independent of the input and capacity is zero. If a physical noise model restricts the transmitter's input frequencies, the maximisation domain must reflect that constraint; the displayed BSC capacity presumes either input bit is permitted with any probability.

Worked Example (Achieved Information Versus Capacity). Suppose $\varepsilon = 0.1$. Then $h_2(0.1) \approx 0.469$. With a uniform input, the achieved mutual information is $1 - 0.469 \approx 0.531$ bits per use, and because the uniform input maximises the BSC expression, the capacity is also approximately 0.531 bits per use. Now consider the same BSC but use $a = 0.1$. The output-one probability is $0.1 + 0.1(0.8) = 0.18$, so

$$I_{0.1}(X;Y) = h_2(0.18) - h_2(0.1) \approx 0.680 - 0.469 = 0.211 \quad \text{bits per use.}$$

The channel has not changed: its capacity remains about 0.531 bits per use. The particular biased input achieves only about 0.211 bits per use. Calling $I_{0.1}(X;Y)$ "the capacity" would confuse a value at one operating distribution with the maximum over distributions. Likewise, the capacity does not mean uncoded bits can be transmitted at that rate with low error. A block code is needed to turn the channel's statistical information supply into reliable messages.

For numerical checks, the following base R function uses vectorised arithmetic and defines the endpoints explicitly, avoiding $\log_2(0)$. It rejects non-finite values and probabilities outside $[0, 1]$. The calculation then reports achieved mutual information and capacity separately for two input laws.

R

```
h2 <- function(p) {
  stopifnot(is.numeric(p), all(is.finite(p)), all(p >= 0 & p <= 1))
  out <- numeric(length(p))
```

```
  interior <- p > 0 & p < 1
  out[interior] <- -p[interior] * log2(p[interior]) -
    (1 - p[interior]) * log2(1 - p[interior])
  out
}

epsilon <- 0.1
a <- c(0.1, 0.5)
output_one <- epsilon + a * (1 - 2 * epsilon)
achieved <- h2(output_one) - h2(epsilon)
capacity <- 1 - h2(epsilon)
data.frame(input_one = a, achieved_bits_per_use = achieved,
           capacity_bits_per_use = rep(capacity, length(a)))
```

Python

```
import math


def h2(p):
    if not math.isfinite(p) or not 0 <= p <= 1:
        raise ValueError("probability must be finite and between zero and one")
    if p == 0 or p == 1:
        return 0.0
    return -p * math.log2(p) - (1 - p) * math.log2(1 - p)


def bsc_mutual_information(input_one, crossover):
    if not math.isfinite(input_one) or not 0 <= input_one <= 1:
        raise ValueError("input probability must be finite and between zero and one")
    noise_entropy = h2(crossover)
    output_one = crossover + input_one * (1 - 2 * crossover)
    return h2(output_one) - noise_entropy


epsilon = 0.1
capacity = 1 - h2(epsilon)
for a in (0.1, 0.5):
    achieved = bsc_mutual_information(a, epsilon)
    print(f"a={a:.1f}: achieved={achieved:.3f}, capacity={capacity:.3f} bits/use")
```

The $a = 1/2$ row reaches capacity; the $a = 0.1$ row does not. Keep those labels in plots and tables. A curve of $I_a(X;Y)$ against a is not a curve of capacity unless each point has itself been maximised over every other permitted channel choice.

6.2.1 Binary Erasure Channel

A BEC with erasure probability δ has output alphabet $\{0, 1, ?\}$. The question mark is an explicit erasure symbol: the receiver knows which position was lost. With probability $1 - \delta$, the input bit is delivered correctly; with probability δ, it is erased. Assume erasures are independent across uses and independent of the input.

Let E indicate whether a position is erased. The output reveals E. If $E = 0$, it reveals X; if $E = 1$, it reveals nothing about X. Consequently, for any input distribution,

$$I(X;Y) = (1-\delta)H(X), \qquad C = 1-\delta \quad \text{bits per use},$$

where capacity uses a uniform input to maximise the binary entropy. One derivation conditions on E: $I(X;Y) = I(X;E) + I(X;Y \mid E)$; independence gives $I(X;E) = 0$, the erased case contributes zero, and the unerased case contributes $H(X)$.

Worked Example. If $\delta = 0.2$, capacity is 0.8 bits per use. In a block of 1,000 uses, the expected number of unerased symbols is 800. This expectation is not a promise of exactly 800 received symbols in each block. The erasure count is random, and even a code that can recover from a stated number of erasures may fail on a larger-than-usual erasure pattern. If a missing packet is silently replaced by a wrong value rather than a visible marker, the BEC model is inappropriate; that event is closer to a substitution error.

The BSC and BEC do not have directly interchangeable crossover parameters. A BSC output is always a bit and can be wrong without announcing the error. A BEC may carry less raw output but tells the decoder which observations are missing. A distance-based code can correct up to $d-1$ known erasures but only $\lfloor (d-1)/2 \rfloor$ arbitrary substitutions, as discussed in Error Correction as Geometry. Those are worst-case guarantees for a code with minimum distance d, not average-error predictions for a random channel.

6.3 Asymmetric Channels and Input Optimisation

Uniform input is optimal for a symmetric channel such as the BSC, but not in general. For a binary asymmetric channel, let $\alpha = P(Y = 1 \mid X = 0)$ and $\beta = P(Y = 0 \mid X = 1)$. The input-one probability is a. Then $P(Y = 1) = \alpha(1-a) + (1-\beta)a$, and

$$I_a(X;Y) = h_2\big(\alpha(1-a) + (1-\beta)a\big) - (1-a)h_2(\alpha) - ah_2(\beta).$$

Capacity is the maximum over $0 \le a \le 1$. The first term rewards an output distribution with entropy; the subtracted terms account for uncertainty introduced

by the channel in each input row. If the channel's two rows differ greatly in reliability, equal input use can waste opportunities by sending too often through the less reliable row.

Worked Example (A Z Channel). Consider a channel where input 0 is always received as 0, while input 1 flips to output 0 with probability $q = 0.2$. Then $\alpha = 0$ and $\beta = 0.2$. If $a = P(X = 1)$,

$$I_a(X;Y) = h_2(0.8a) - ah_2(0.2).$$

At $a = 1/2$, this is $h_2(0.4) - 0.5h_2(0.2) \approx 0.971 - 0.361 = 0.610$ bits per use. The optimum is not at one half. Differentiating in the interior gives

$$0.8 \log_2 \frac{1 - 0.8a}{0.8a} = h_2(0.2),$$

which yields $a \approx 0.436$ and capacity about 0.619 bits per use. The numerical improvement is modest here, but the example establishes the principle: capacity is an optimisation over the input law, not a fixed entropy attached to the transition table. In other channels the difference can be substantial.

For a finite alphabet, this is a concave optimisation problem in the input probabilities, so standard numerical optimisation is available. At the boundary, probabilities of zero must be treated by continuity. The resulting optimiser can be non-unique for some channels; if so, the capacity value is unique even though multiple input laws achieve it. A learned or estimated input law is not automatically the maximiser: it describes observed traffic, whereas capacity asks what the channel could support under the allowed input choices.

6.4 Capacity and the Channel Coding Theorem

For a finite DMC W, its Shannon capacity is

$$C = \max_{p(x)} I_p(X;Y) \quad \text{bits per channel use.}$$

This is an optimisation over the channel input distribution. To interpret capacity operationally, define a block code with M_n possible messages, an encoder mapping messages to length-n input sequences, and a decoder mapping received

sequences to message estimates. Its rate is $R_n = (1/n)\log_2 M_n$ bits per channel use. Under a specified channel law, its average error probability is the probability that a uniformly selected message is decoded incorrectly, averaged over messages and channel noise.

Theorem (DMC Channel Coding Theorem). Let $W(y \mid x)$ be a fixed finite-alphabet discrete memoryless channel with no feedback, and let $C = \max_{p(x)} I_p(X;Y)$. For every $R < C$, there is a sequence of length-n block codes with rates approaching R and average decoding error tending to zero as $n \to \infty$. For every $R > C$, no sequence of such codes can have rate approaching R while its average error tends to zero. The boundary rate $R = C$ is not asserted achievable with vanishing error by this statement. A maximal-error formulation requires care about the channel and conventions; the statement here uses average error and a uniform message.

Proof Sketch. For achievability, fix an input law whose mutual information exceeds R. Generate many candidate codewords at random according to its product distribution. For a long block, the transmitted input and output are jointly typical with high probability. A decoder declares the unique codeword jointly typical with the observed output. A competing independently generated codeword has probability roughly $2^{-nI(X;Y)}$ of passing this test; with about 2^{nR} candidates, the union bound tends to zero when $R < I(X;Y)$. Taking input laws closer to the capacity maximiser gives all rates below C. This is an existence proof; random generation does not itself yield a conveniently structured or efficiently decoded code.

For the converse, let M be the message and $\widehat{M}$ its decoded estimate, with error probability P_e. Fano's inequality bounds $H(M \mid \widehat{M})$ by $h_2(P_e)+P_e \log_2(M_n-1)$. Therefore reliable decoding would require the output to carry nearly $\log_2 M_n$ bits of information about the message. The encoder and channel form a Markov chain from message to input sequence to output sequence, and memorylessness together with the chain rule bounds the information conveyed by nC. A rate above C cannot have vanishing error. The omitted technical steps formalise typicality and the limiting inequalities; see Further Reading for complete proofs.

The theorem's qualifications matter. It assumes a known, fixed channel law and permits arbitrarily long blocks. It makes no claim about a chosen short block, code family, decoder complexity, delay, energy use, or channel-estimation error. It does not say that a source whose entropy is below C can be sent at its entropy

without a source encoder, nor that capacity is bits per second. If the system permits r channel uses per second, an ideal asymptotic information rate is rC bits per second; protocol overhead and implementation loss reduce delivered application throughput.

At finite block length, rate and error probability form a trade-off rather than a sharp operational step at capacity. A system designer chooses a block length n, a message count M_n, and an acceptable error target. The rate is $(\log_2 M_n)/n$, but the theorem does not say how large n must be before the error becomes small. A capacity-achieving family can have poor error at the block lengths allowed by a low-latency application. Finite-blocklength bounds refine the asymptotic answer, but require additional analysis and often depend on channel dispersion as well as capacity.

The observed empirical error from a test transmission is also not the channel capacity. A low error rate may reflect a very low code rate, a favourable finite noise realisation, or a test channel that differs from deployment. Conversely, an error on one codeword does not establish that the rate exceeds capacity. Report the code rate, block length, decoder, test protocol, sample count, and observed error with an uncertainty interval; report a model-based capacity separately as a theoretical benchmark. When the channel parameters themselves are estimated, their uncertainty propagates into the capacity calculation. For example, the BSC capacity $1 - h_2(\varepsilon)$ is sensitive to the estimated crossover rate, especially near the region where the entropy curve changes rapidly.

Worked Example (Repetition Is Not Capacity). Send a bit three times through a BSC with $\varepsilon = 0.1$ and decode by majority vote. Decoding fails if at least two of the three uses flip:

$$P_e = \binom{3}{2}(0.1)^2(0.9) + (0.1)^3 = 0.028.$$

The code rate is $1/3$ message bit per channel use. Its error probability is lower than the uncoded bit error probability of 0.1, at the cost of using three times as many channel uses. The BSC capacity is about 0.531 bits per use, but this repetition code operates at 0.333 bits per use with a non-zero finite-block error. Neither number is its achieved mutual information under a source bit distribution; they answer different questions. This is a useful small illustration, not a

capacity-approaching construction.

6.4.1 Error Correction as Geometry

A binary block code can be viewed as a set of length-n binary strings. The Hamming distance $d_{\mathrm{H}}(u, v)$ counts positions at which two strings differ. If the minimum distance between distinct codewords is d, then Hamming balls of radius $t = \lfloor (d-1)/2 \rfloor$ around them do not overlap. Nearest-codeword decoding therefore corrects every pattern of at most t substitutions. The qualifier "every" makes this a worst-case property of the code; it does not give the probability that a memoryless channel produces more than t flips.

For the threefold repetition code, the codewords are "000" and "111", at distance three. Their radius-one balls are disjoint, so majority decoding corrects any single flipped bit. On a BSC with crossover 0.1, two or three flips occur with probability 0.028, exactly the block error calculated above. The geometry supplies the correction threshold, while the channel law supplies the probability of crossing it. If bit flips are correlated, the geometric threshold remains true but the binomial probability formula no longer applies.

Known erasures are easier to locate. If a code has minimum distance d, two codewords cannot agree outside fewer than d erased positions, so any pattern of at most $d - 1$ known erasures has a unique compatible codeword. For the repetition code, two erasures still leave one bit and can be corrected; three erasures leave no information. This comparison explains why a BEC and BSC with the same numerical error parameter have different capacities and decoding problems. It does not imply that minimum distance alone predicts average-error performance: the number and placement of nearby codewords, the decoder, and the channel probabilities matter.

Long practical codes trade rate, block length, error probability, and decoder work. A parity-check code may make error patterns detectable without making all of them correctable. A cyclic redundancy check can flag many altered packets and support retransmission, yet a detected error is not a decoded message. Report whether a claimed reliability figure is raw symbol error, codeword error, undetected error, or application-level loss after retransmissions.

6.5 The Real AWGN Channel

For a real-valued signal, an often-used ideal model is $Y = X + Z$, where $Z \sim \mathcal{N}(0, N)$ is independent of X on each use. Impose an average input-power constraint $\mathbb{E}[X^2] \leq P$. The capacity is

$$C = \frac{1}{2} \log_2 \left(1 + \frac{P}{N}\right) \quad \text{bits per real channel use.}$$

The formula assumes a real scalar channel used repeatedly with independent Gaussian noise of fixed variance, an average power constraint, no additional bandwidth or peak-amplitude restriction, and coding over arbitrarily long blocks. It is not a universal formula for any system that happens to contain Gaussian noise. If $P = 0$, capacity is zero; increasing the signal-to-noise ratio P/N increases capacity logarithmically. Doubling power does not double capacity.

The achievability intuition uses a Gaussian input with variance P, which maximises differential entropy among real distributions with that variance. For any admissible input, $h(Y) \leq \frac{1}{2} \log_2(2\pi e(P + N))$, while independent Gaussian noise has $h(Z) = \frac{1}{2} \log_2(2\pi eN)$. Since $I(X;Y) = h(Y) - h(Z)$, their difference gives the stated upper bound and is attained by the Gaussian input under the ideal model. A complete proof must also handle block codes and the power constraint; this entropy argument explains the single-use optimisation.

For example, if $P/N = 3$, capacity is $\frac{1}{2} \log_2(4) = 1$ bit per real use. This is not one bit on every use, and a practical finite-block system may need a margin below it. A complex baseband convention often has no factor $1/2$ because each complex use represents two real dimensions; the unit convention must be stated before comparing formulas. Bandwidth, sampling, and spectral-efficiency conversions need additional definitions. Coloured noise, fading, peak-power limits, non-Gaussian noise, and channel memory require a different optimisation or theorem.

The average-power condition is a property of the allowed ensemble or code sequence, according to the chosen operational formulation. It does not mean every transmitted sample has squared magnitude at most P. A Gaussian input has unbounded tails and would violate a strict peak-amplitude bound, even though its average squared value is P. Under a peak constraint, the capacity-achieving input need not be Gaussian and the displayed formula should not be reused

without proof. Likewise, “white” in this scalar memoryless model means independent, identically distributed noise across uses; a time-correlated Gaussian process cannot simply be assigned the same per-use capacity from its marginal variance.

Compare units before interpreting a measured link. A sample rate of r real channel uses per second gives the ideal ceiling rC bits per second under this model. If bandwidth B hertz is used to justify a sample rate, the relationship between B, real dimensions, and signalling convention must be stated. Framing, pilot symbols, modulation restrictions, finite codes, and protocol retransmissions can reduce application throughput. A reported signal-to-noise ratio also needs a reference point: receiver input power, post-filter noise variance, and per-symbol energy are not interchangeable without a system model.

7 Lossy Compression and Rate-Distortion

Lossless coding tries to reproduce every source symbol exactly. Many tasks instead permit controlled error: an image can lose detail, a measurement can be rounded, or a stored signal can be approximated. The word *loss* is not yet a mathematical model. We must define which source and reconstruction pairs count as costly, then ask how many bits are needed at a chosen average cost.

Rate–distortion theory formalises this choice. Its answer depends on the source distribution and distortion measure, not only on the source alphabet. The central function is an asymptotic bound for long blocks of a memoryless source; it is not an objective quality score that automatically matches human judgement.

7.1 Distortion and the Rate–Distortion Function

Let X be a source symbol and $\widehat{X}$ its reconstruction. A distortion function $d(x, \hat{x}) \geq 0$ assigns a cost to each pair. Under a conditional reconstruction law $p(\hat{x} \mid x)$, expected distortion is $\mathbb{E}[d(X, \widehat{X})]$. For a target $D \geq 0$, define

$$R(D) = \min_{p(\widehat{x}|x):\, \mathbb{E}[d(X,\widehat{X})] \leq D} I(X; \widehat{X}).$$

The minimisation ranges over all conditional distributions that meet the distortion constraint, with the source marginal fixed. Mutual information measures

how much the reconstruction must reveal about the source under that chosen reproduction channel. As D increases, the feasible set expands and $R(D)$ cannot increase. Typically $R(D)$ is convex and non-increasing; convexity follows by time-sharing between two reconstruction channels and convexity of mutual information in the channel for a fixed input law.

Theorem (Rate–Distortion, Discrete Memoryless Form). Let $X_1, X_2, \ldots$ be independent and identically distributed from a finite-alphabet law $p(x)$, and let $d(x, \hat{x})$ be a bounded single-letter distortion measure on finite source and reproduction alphabets. For any distortion level D, rates strictly above $R(D)$ permit length-n lossy codes whose expected per-symbol distortion is at most D plus arbitrarily small slack for sufficiently large n. Rates strictly below $R(D)$ cannot meet that asymptotic expected-distortion target. This statement is in the block-length limit and concerns expectation under the source model; it does not guarantee a maximum distortion for every block. Boundary and zero-probability cases need the usual limiting interpretation.

Proof Sketch. For achievability, choose a conditional law $p(\hat{x} \mid x)$ meeting the distortion constraint. Randomly construct a codebook of reproduction sequences drawn from the induced marginal $p(\hat{x})$. Typicality arguments show that a codebook with rate slightly greater than $I(X; \widehat{X})$ contains, with high probability, a sequence jointly typical with the source block. The encoder selects one such sequence, and typical average distortion approaches the chosen expectation. For the converse, any encoder and decoder induce a joint distribution between source block and reconstruction block. The data-processing inequality and chain rule bound the rate needed for that dependence below by the sum of single-letter mutual-information terms; convexity reduces this to at least $nR(D)$ when average distortion is at most D. The full argument is longer than this sketch because it controls atypical blocks and converts a block code into single-letter distributions.

The rate is bits per source symbol, not bits per reconstruction symbol or per second. The distortion is also per source symbol. If a sensor emits 100 samples per second and the rate–distortion point is 0.7 bits per sample, the ideal asymptotic description rate is 70 bits per second before framing and model overhead. If a design instead specifies a limit on the worst individual sample error, expected-distortion theory does not directly answer it.

7.2 Binary Hamming Distortion

For a binary source with $X \sim \text{Bernoulli}(p)$, use Hamming distortion $d(x, \hat{x}) = \mathbf{1}\{x \neq \hat{x}\}$. This counts the fraction of incorrect reconstructed bits. If $p = 1/2$, the rate–distortion function is

$$R(D) = 1 - h_2(D), \quad 0 \leq D \leq \tfrac{1}{2}; \qquad R(D) = 0, \quad D \geq \tfrac{1}{2}.$$

For $D = 0$, exact reconstruction requires one bit per fair source bit. At $D = 0.1$, the asymptotic minimum is $1 - h_2(0.1) \approx 0.531$ bits per source bit. At $D \geq 1/2$, a decoder can always output a fixed bit and incur error probability one half, so no information need be sent. Distortion greater than one half has no benefit under this measure because the decoder can invert its fixed output if needed.

The binary formula has a short information argument. Write the reconstruction error as $E = X \oplus \widehat{X}$. Since X is determined by $(\widehat{X}, E)$, the remaining uncertainty satisfies $H(X \mid \widehat{X}) \leq H(E) \leq h_2(D)$ when the error probability is at most $D \leq 1/2$. For a fair source, $H(X) = 1$, so $I(X; \widehat{X}) \geq 1 - h_2(D)$. Equality can be attained by choosing a fair reconstruction bit and passing it through an independent binary flip of probability D to produce X; this construction has a fair source marginal and error probability D. The backwards description is a test channel for the optimisation, not an instruction that a real encoder knows the reconstruction before seeing its source bit.

A distortion target of $D = 0.1$ is an expectation over the source and, for a randomised code, the coding randomness. It permits some blocks to have more than 10% incorrect bits. A service contract requiring every delivered block to have at most 10% errors imposes a different, stronger criterion. At a finite length, even a code whose expected error fraction is 0.1 can produce an unusually bad block. A tail probability or worst-case bound needs separate analysis.

The uniform-source assumption matters for the displayed $1 - h_2(D)$ curve. For $X \sim \text{Bernoulli}(p)$ with Hamming distortion, the corresponding finite-alphabet result is $R(D) = h_2(p) - h_2(D)$ for $0 \leq D \leq \min(p, 1-p)$, and zero for $D \geq \min(p, 1-p)$. If $p = 0.1$, always reconstructing zero already incurs distortion 0.1 at zero rate. At $D = 0.05$, the rate is $h_2(0.1) - h_2(0.05) \approx 0.183$ bits per source symbol, much smaller than the fair-source value. The rate–distortion curve reflects the actual source law; a statement of "binary rate–distortion"

without both the distribution and distortion measure is incomplete.

7.3 Gaussian Squared-Error Distortion

Let X be an independent and identically distributed Gaussian source with mean zero and variance σ^2. Use squared-error distortion $d(x, \hat{x}) = (x - \hat{x})^2$. Then

$$R(D) = \begin{cases} \frac{1}{2}\log_2(\sigma^2/D), & 0 < D < \sigma^2, \\ 0, & D \geq \sigma^2. \end{cases}$$

The formula is an asymptotic rate in bits per real-valued source sample. At $D = \sigma^2$, the constant reconstruction $\widehat{X} = 0$ already has expected squared error σ^2, so zero rate suffices. As $D \downarrow 0$, the required rate diverges. For a Gaussian source the exact zero-distortion point is not a finite-rate point because the source has a continuum of values.

To see the lower-bound mechanism, suppose a reconstruction has mean-squared error at most D. The error entropy is maximised by a Gaussian at fixed variance, while the source-reconstruction mutual information is at least the difference between source entropy and that conditional error entropy. This yields $I(X; \widehat{X}) \geq \frac{1}{2}\log_2(\sigma^2/D)$. A Gaussian test channel with appropriately chosen independent Gaussian error attains the bound. The full coding theorem turns this single-letter optimisation into achievable long-block compression.

For the attaining test channel at $0 < D < \sigma^2$, take an independent Gaussian error $E \sim \mathcal{N}(0, D)$ and a reconstruction $\widehat{X} \sim \mathcal{N}(0, \sigma^2 - D)$, then set $X = \widehat{X} + E$. The resulting X has the required $\mathcal{N}(0, \sigma^2)$ law and expected squared error D. Moreover $h(X \mid \widehat{X}) = h(E)$, so $I(X; \widehat{X}) = \frac{1}{2}\log_2(\sigma^2/D)$. Describing the test channel in this direction avoids an easy mistake: adding independent noise of variance D *to* an already fixed Gaussian source gives a different mutual information, $\frac{1}{2}\log_2(1 + \sigma^2/D)$, and does not attain the rate–distortion bound.

The source mean is irrelevant if it is known to both sides: subtract it before coding and add it back after reconstruction. If the mean or variance must be learned and transmitted, their description and estimation errors are extra costs. The iid hypothesis is more consequential. A correlated Gaussian sequence can be compressed by exploiting predictability, and its rate–distortion function involves the source spectrum rather than only one marginal variance. Conversely,

a non-Gaussian source with the same variance does not inherit the Gaussian formula merely because squared error is used.

Worked example (rate, block length, and error). Suppose an iid Gaussian sensor has variance 9 and the desired expected squared error is 1. The ideal asymptotic rate is $\frac{1}{2}\log_2 9 \approx 1.585$ bits per sample. For 1,000 samples, multiplying suggests roughly 1,585 description bits before finite-length overhead, but the theorem does not promise a 1,585-bit file or a block error bounded by one unit. If the sensor emits 200 samples each second, the corresponding asymptotic source-information rate is about 317 bits per second. A physical link must then supply enough channel uses and reliability; comparing 1.585 bits per sample directly with capacity in bits per channel use would mix units.

This expression does not describe perceptual image quality, a bounded sensor, a heavy-tailed distribution, or a signal with temporal dependence without additional modelling. Squared error heavily penalises large errors and ignores semantic structure. A more appropriate distortion could weight frequency bands, count classification mistakes, or encode domain-specific consequences; then the rate–distortion curve changes. Distortion is a modelling decision, and reporting $R(D)$ without identifying it conceals the main value judgement.

7.4 Choosing a Distortion Measure

For categorical records, zero-one loss may be understandable, but confusing two categories may have very different costs. For a medical measurement, squared error gives large deviations disproportionate influence, while absolute error is less sensitive to outliers. For images and audio, pixelwise or samplewise error can disagree with perceptual similarity. A distortion measure should describe the consequence that the application is prepared to tolerate, and it should be checked against examples that domain users recognise as acceptable and unacceptable.

Rate–distortion theory does not decide those consequences. It tells us the best asymptotic information rate under the chosen distribution and cost. A codec can optimise a proxy faithfully while producing outputs users judge poor if the proxy is wrong. Conversely, a complex perceptual metric may be difficult to estimate and optimise. State both the mathematical criterion and the validation procedure rather than treating one as a substitute for the other.

An operational comparison helps prevent unit errors:

Question	Quantity	Units	Governing choice
How much source uncertainty is present?	$H(X)$	bits per source symbol	Source law and alphabet
How much information does a selected channel input convey?	$I(X;Y)$	bits per channel use	Input law and channel law
What is the best asymptotic reliable channel rate?	C	bits per channel use	Channel law and allowed inputs
How many bits permit a stated average loss?	$R(D)$	bits per source symbol	Source law and distortion measure
How many bits cross a system per second?	Throughput	bits per second	Uses per second, coding, overhead, and implementation

8 Estimating Information from Data

The definitions above concern a probability model. In applied work, the model is usually unknown and the analyst has a finite sample. Replacing probabilities by observed frequencies is a natural first step, but it does not remove uncertainty. Estimators have bias and variance, sparse outcomes are easily missed, dependence invalidates independent-sample formulas, and analysis choices can inflate apparent dependence.

The appropriate question is not merely “What is the mutual information?” It is “Which population quantity, under which sampling model, is this estimate intended to approximate, and how uncertain is it?” This section develops a basic workflow for discrete data and indicates where more specialised estimators are

needed.

8.1 Plug-In Estimates and Their Bias

Suppose $X_1, \ldots, X_n$ are independent observations from a fixed distribution on k possible categories. Let N_x be the number of observations equal to x, and define $\hat{p}(x) = N_x/n$. The plug-in entropy estimator is

$$\widehat{H}(X) = -\sum_x \hat{p}(x) \log_2 \hat{p}(x),$$

with zero-count terms set to zero. It is consistent for fixed finite support: as n grows, empirical frequencies converge to their probabilities and the plug-in entropy converges to $H(X)$. For finite samples, it is typically downward-biased. An outcome that exists in the population but was not sampled appears to have probability zero, so its uncertainty is omitted; the concavity of entropy also contributes to the bias.

For fixed k with all k probabilities positive and n sufficiently large, the leading-order bias is approximately

$$\mathbb{E}[\widehat{H}] - H(X) \approx -\frac{k-1}{2n \ln 2}.$$

This Miller–Madow approximation adds $(k-1)/(2n \ln 2)$ to the plug-in estimate when the support is known and the regular large-sample conditions are reasonable. It is not a universal fix. If the alphabet grows with sample size, probabilities are extremely small, observations are dependent, or the actual support is unknown, the approximation can be poor. Smoothing, Bayesian estimators, coverage-based methods, and bias-corrected estimators each introduce assumptions; there is no estimator that makes sparse data information-free.

Worked Example (A Missing Category). Suppose a status sensor can report four states, with true probabilities $(0.7, 0.2, 0.08, 0.02)$. In a sample of 20 readings, it is quite possible not to observe the rarest state. The empirical support then has only three observed states, and its plug-in entropy treats the absent fourth state as impossible. Even if it is observed once, the frequency estimate $1/20 = 0.05$ is noisy relative to the true probability 0.02. This example shows

why reporting only a decimal entropy without the sample size, observed category counts, and support assumptions hides uncertainty.

When categories are known in advance, preserve unobserved levels in the table rather than silently dropping them. In R, `table(factor(x, levels = ...))` helps retain structural zeros; distinguish those from sampling zeros. A structural zero is impossible by design under the model. A sampling zero is an unobserved event that may still have positive population probability. Adding a pseudocount treats the latter as possible, but the pseudocount is a modelling choice rather than evidence.

8.2 Joint Tables, Sparse Cells, and Mutual Information

For paired observations (X_i, Y_i), empirical joint frequencies give a plug-in mutual information estimate:

$$\hat{I}(X;Y) = \sum_{x,y} \hat{p}(x,y) \log_2 \frac{\hat{p}(x,y)}{\hat{p}_X(x)\hat{p}_Y(y)}.$$

The estimate is non-negative by construction, even when the population variables are independent. Random empirical imbalances create apparent dependence; under the null, the plug-in mutual information is generally upward-biased. The bias increases with the number of populated degrees of freedom relative to sample size. A table with r rows and c columns has $(r-1)(c-1)$ interaction degrees of freedom in the regular interior case, so a sparse $r \times c$ table can exaggerate the problem.

The approximation

$$2n \ln 2 \, \hat{I}(X;Y) \approx \chi^2_{(r-1)(c-1)}$$

is useful as a large-sample null reference for an independence test when expected cell counts are adequate and the usual regularity conditions hold. It is not a confidence interval for the information value and should not be trusted automatically when many expected counts are small. Pooling categories can be defensible when done for substantive reasons before examining the association; pooling solely to obtain a desired result biases the analysis. Exact or Monte

Carlo tests can handle some sparse settings, but they still answer a test question under a specified sampling design.

Worked Example (Finite-Sample Association). Let two independent fair binary variables be sampled in 40 pairs. Even though their population mutual information is zero, the four cell counts will rarely all equal ten. The plug-in estimator will therefore usually be positive. If the observed counts are $12, 8, 8, 12$, the empirical table has equal marginals and

$$\widehat{I} = 2\left(0.3\log_2\frac{0.3}{0.25} + 0.2\log_2\frac{0.2}{0.25}\right) \approx 0.029 \text{ bits per pair.}$$

This number describes empirical dependence in this table. It is not proof that the population variables share information. A null comparison that repeats the sampling process clarifies how unusual a value of this size would be under independence.

The following base R function computes the plug-in value from a contingency table. It requires a numeric matrix or two-way table of finite, non-negative counts with a positive, finite total. Matrix margins keep the marginals explicit, and logical indexing selects only positive cells, so zero cells contribute zero. The Python implementation uses NumPy arrays with the same validation; `axis=1` sums across columns to leave row totals, and `axis=0` leaves column totals.

R

```r
mi_bits <- function(counts) {
  stopifnot(is.matrix(counts), is.numeric(counts), all(is.finite(counts)),
            all(counts >= 0), is.finite(sum(counts)), sum(counts) > 0)
  joint <- counts / sum(counts)
  expected <- outer(rowSums(joint), colSums(joint))
  occupied <- joint > 0
  sum(joint[occupied] * log2(joint[occupied] / expected[occupied]))
}

counts <- matrix(c(12, 8, 8, 12), nrow = 2, byrow = TRUE)
mi_bits(counts)
```

Python

```python
import numpy as np


def empirical_mi_bits(counts):
    counts = np.asarray(counts, dtype=float)
    if counts.ndim != 2 or counts.size == 0:
        raise ValueError("counts must be a non-empty two-dimensional table")
    if not np.all(np.isfinite(counts)) or np.any(counts < 0):
        raise ValueError("counts must be finite and non-negative")
    total = counts.sum()
    if not np.isfinite(total) or total <= 0:
        raise ValueError("counts must have a positive, finite total")
    joint = counts / total
    px = joint.sum(axis=1, keepdims=True)
    py = joint.sum(axis=0, keepdims=True)
    expected = px @ py
    occupied = joint > 0
    return float(np.sum(joint[occupied] * np.log2(
        joint[occupied] / expected[occupied]
    )))


counts = np.array([[12, 8], [8, 12]])
print(f"empirical MI: {empirical_mi_bits(counts):.3f} bits/pair")
```

The result is approximately 0.029 bits per pair. This function returns a point estimate only. It does not account for the sampling design, missing data, repeated subjects, selection of the variables after looking at the data, or uncertainty in categories and preprocessing. Add those parts to the analysis plan.

For raw categorical vectors, base R can produce the joint proportions without hand-entering a matrix. Reuse `mi_bits()` from the preceding block. The factor

levels state the planned support; unused levels remain visible as zero-count cells. The orientation of the table is explicit: rows are source categories and columns are observed categories.

```
source_bit <- factor(c(0, 0, 0, 1, 1, 1), levels = c(0, 1))
observed_bit <- factor(c(0, 0, 1, 0, 1, 1), levels = c(0, 1))
counts <- table(source_bit, observed_bit)
joint <- prop.table(counts)
stopifnot(abs(sum(joint) - 1) < 1e-12)
mi_bits(counts)
```

Here `table()` records counts and `prop.table()` divides by their total. The `mi_bits()` function normalises internally, so passing `joint` instead of `counts` gives the same numerical answer, but the counts should be retained for interpreting sampling error. A zero in this table means no observation in that cell. It does not prove that the population transition probability is zero. If a level was impossible by design, document that separately and exclude it from models that require positive support.

We can expose the upward bias under independence with repeated simulated samples, again using `mi_bits()`. This simulation fixes independent fair population variables and repeats the same estimator, so its distribution measures finite-sample behaviour under a known null rather than uncertainty about an unknown real dataset.

```
set.seed(163)
null_mi <- replicate(1000, {
  x <- factor(sample(0:1, 40, replace = TRUE), levels = 0:1)
  y <- factor(sample(0:1, 40, replace = TRUE), levels = 0:1)
  mi_bits(table(x, y))
})
c(mean = mean(null_mi), median = median(null_mi),
  upper_95_percentile = unname(quantile(null_mi, 0.95)))
```

The population MI is exactly zero, while the simulated estimates are normally positive. The exact numerical summaries depend on the random seed and the sample size. The simulation has only two categories per variable; a table with many possible cells and the same 40 pairs has a harder estimation problem. Repeating a computation 1,000 times makes Monte Carlo error smaller, but it does not fix an estimator's bias.

The same resampling idea answers different questions depending on what is regenerated. Independently sampling both variables from their known laws, as above, studies a synthetic null. Permuting the observed `Y` labels conditions on the empirical marginals and tests exchangeability of pairings. Bootstrapping the observed pairs approximates variation under their empirical joint law. Substituting one for another changes the inferential target. If the data contain re-

peated measurements from one person, resampling individual rows also gives misleadingly narrow uncertainty because the rows are not independent units.

8.3 Permutation Tests for Dependence

A permutation test can estimate the null distribution of empirical mutual information under independence while preserving the observed marginal counts. Keep the X values fixed and randomly permute the Y labels across independent paired observations. Each permutation breaks the pairing but leaves each variable's empirical marginal unchanged. Recompute the entire statistic for each shuffled data set. If B random permutations produce statistics $T_1^*, \ldots, T_B^*$ and the observed statistic is T_{obs}, a conservative Monte Carlo p-value is

$$\hat{p} = \frac{1 + \sum_{b=1}^{B} \mathbf{1}\{T_b^* \geq T_{\text{obs}}\}}{B + 1}.$$

The added one avoids a zero p-value and gives a valid Monte Carlo test convention under exchangeability. Permutation tests assess a null hypothesis; they do not directly estimate a bias-corrected population mutual information or provide an effect-size interval. A tiny p-value can accompany a practically negligible association in a huge sample.

The permutation is valid only if pairings are exchangeable under the null. If observations are clustered by person, site, time block, or experimental unit, free permutation breaks the design. Permute at the appropriate unit, use restricted permutations, or fit a model that respects the dependence. For time series, naive shuffling destroys serial structure; use a null procedure designed for that process. The mechanics of shuffling are simple, but exchangeability is a substantive assumption.

8.3.1 Bootstrap Uncertainty and Resampling Units

The non-parametric bootstrap resamples observed units with replacement and recomputes the estimator. If the data are independent pairs, resample pairs (X_i, Y_i), not the two columns separately; resampling columns separately changes the joint distribution. Quantiles of bootstrap estimates give an uncertainty interval when the estimator's sampling distribution is sufficiently well represented by resampling the observed data.

Entropy and mutual information are nonlinear functions, so bootstrap intervals can behave poorly near boundaries, with many zero cells, or when rare categories are not observed at all. A bootstrap cannot recreate outcomes missing from every resample's parent data. For very sparse tables, a parametric bootstrap from a fitted model, a Bayesian posterior, or a specialised coverage method may be more defensible, each with its own assumptions. Report the resampling unit, interval method, number of resamples, and treatment of unobserved categories.

Permutation and bootstrap answer different questions. Permutation constructs a reference distribution under a null such as independence. Bootstrap estimates sampling variability around the empirical distribution. Neither should be described generically as "the uncertainty" without stating the target and procedure. For repeated measures, both must respect subject or cluster structure.

8.4 Continuous Variables and Estimator Choice

For continuous variables, discretising into bins changes the estimand. Mutual information between binned X and binned Y is the information between those categorical versions, and changing the bin edges can change the result. Differential entropy is not a drop-in replacement: as explained in Continuous Entropy and Relative Entropy, it depends on units and can be negative. Mutual information is invariant to smooth invertible changes of coordinates under suitable conditions, but estimating it from a finite sample remains difficult.

Common approaches include nearest-neighbour estimators, kernel methods, density-ratio estimation, and model-based likelihood ratios. They trade assumptions about smoothness, dimensionality, and computation. High-dimensional estimates can be unstable because local neighbourhoods become sparse. A model-based estimate can be precise under a correct model and misleading under a wrong one. Validate with simulated data whose dependence is known, inspect sensitivity to tuning parameters, and state whether the target is discrete MI, continuous MI, or MI after a declared discretisation.

There is no universal rule that "more bins" or "more data" alone solves the problem. Increasing resolution while holding sample size fixed creates sparse cells; increasing sample size under a drifting distribution violates the fixed-target assumption. Decide what random variables and population are being studied before selecting the estimator.

8.4.1 Reporting an Estimate so It Can Be Reused

A useful report makes the calculation reproducible at the level of assumptions even if the raw data cannot be shared. State the target population and sampling period; define each random variable and its coding; give the number of independent sampling units and the number of repeated measurements; and list the category counts or a sufficient summary of the support. If data were filtered, weighted, imputed, or grouped, describe the operation and whether it was specified before examining the association.

For entropy, name the estimator and say whether the support size was fixed in advance, inferred from observed values, or regularised. For mutual information, report the joint and marginal support sizes, the number of occupied cells, and whether the statistic is a plug-in estimate, model-based estimate, or estimator for a discretised version of continuous data. For uncertainty, state whether an interval comes from a bootstrap, an asymptotic approximation, a posterior, or another method. For a test, state the null, test statistic, number of permutations, and exchangeability unit. These details often matter more than the fourth decimal place.

A compact report might read: “For 1,240 independent device-days, plug-in MI between the four-level alarm state and the binary service outcome was 0.083 bits per device-day. The joint table had six occupied cells of eight possible. A 5,000-replicate bootstrap of device-days gave a percentile interval of [0.061, 0.108] bits. Under a pre-specified independence null, 9,999 permutations of outcomes across device-days gave Monte Carlo $p = 0.004$. The estimate is descriptive of monitored devices during the stated quarter; it does not identify a causal effect.” This is only a reporting pattern, not a claim that percentile intervals or unrestricted permutations suit every design. If there are multiple days per device, resampling and permutation must instead respect the device clustering.

Pre-registration is especially helpful when many variable pairs, bin edges, smoothing parameters, or subgroups could be explored. If exploration has already occurred, say so and validate the chosen estimate on new data where possible. The selection process changes the distribution of the reported maximum association. A nominal interval for one fixed estimator does not account for screening thousands of candidates and publishing only the largest result.

Finally, preserve the distinction between uncertainty about the estimate and un-

certainty about the model. A narrow confidence interval can quantify sampling variability conditional on an iid assumption while the observations are actually clustered or the population is shifting. Sensitivity analysis over defensible sampling and modelling assumptions is often more informative than a more elaborate interval under one fragile assumption.

8.4.2 A Practical Estimation Workflow

For a defensible estimate, write down the estimand in plain language first: for example, “mutual information in bits between a binary alarm and a four-level sensor category in the target population of devices during normal operation”. Then identify the sampling unit and how observations entered the sample. Check whether the sample is independent, clustered, weighted, time-dependent, or selected based on the outcome. Construct the table with all design-known categories, marking structural and sampling zeros separately.

Next calculate a transparent plug-in estimate as a baseline, but accompany it with sample size, support sizes, occupied cells, and an uncertainty or null procedure suited to the question. Use a permutation test for a null of independence when exchangeability is justified; use bootstrap resampling of the correct units for sampling uncertainty. For sparse cells, show sensitivity to a justified smoothing or model choice rather than presenting one correction as truth. For continuous data, define the estimator and its tuning choices before reading the result.

Finally, compare the result with a task-relevant baseline. If the intended use is prediction, held-out predictive performance may answer the practical question better than a global information estimate. If the intended use is compression, evaluate held-out code length under the stated model. A statistic becomes useful when its population meaning and decision role are clear; a precise decimal alone does not establish either.

9 Connections to Statistics and Machine Learning

Information theory supplies quantities that recur in statistical modelling: likelihood, log loss, divergence, dependence, and compression. They can clarify what a learning objective rewards, but they do not remove choices about targets, populations, costs, or validation. This section connects earlier Cross-Entropy and

Log Loss, Mutual Information, and Data Processing Inequality to prediction and representation learning.

9.1 Log Loss, Cross-Entropy, and KL Direction

For a categorical outcome Y and a probabilistic prediction $q(y \mid x)$, logarithmic loss on an observed pair (x, y) is $-\log_2 q(y \mid x)$. Its expected value under the true data distribution $p(x, y)$ is the conditional cross-entropy

$$H_p(Y \mid X; q) = -\sum_{x,y} p(x, y) \log_2 q(y \mid x).$$

Condition on each x and apply the cross-entropy decomposition from Cross-Entropy and Log Loss:

$$H_p(Y \mid X; q) = H_p(Y \mid X) + \mathbb{E}_{X \sim p}\left[D_{\mathrm{KL}}(p(\cdot \mid X) \| q(\cdot \mid X))\right].$$

The direction matters. The data-generating conditional distribution is first, the model's predicted distribution second. The expected excess log loss is $D_{\mathrm{KL}}(p\|q)$, not generally $D_{\mathrm{KL}}(q\|p)$. If the model assigns zero probability to an outcome that can occur, its log loss is infinite. In practice, softmax models usually assign positive mass, though numerical clipping changes the literal objective.

When the evaluation distribution is fixed, minimising expected log loss over a sufficiently rich model class minimises this conditional KL divergence. With limited data, regularisation and architecture constrain the class; training log loss is a sample estimate and may improve while held-out log loss worsens. This connects to source coding: a probabilistic predictor induces ideal code lengths $-\log_2 q(y \mid x)$, and the expected penalty for a mismatched predictor is conditional KL. This interpretation does not make a model calibrated or useful for every decision by itself.

Worked example (a forecast versus the marginal baseline). Suppose rain occurs on 20% of days in a stable target population. A predictor that always assigns 0.2 to rain has expected log loss $h_2(0.2) \approx 0.722$ bits per day. A feature-informed predictor can improve only if its conditional probabilities capture real dependence: the best possible population expected log loss using feature X is $H(Y \mid X)$, and the largest possible improvement over the marginal baseline is

$H(Y) - H(Y \mid X) = I(X;Y)$. This identity assumes both predictors know the true distributions. A fitted model with limited data may have worse held-out loss than the marginal forecast even when population MI is positive.

Suppose instead the forecaster always reports 0.01 for rain in that 20% population. Expected log loss rises to $-0.2\log_2(0.01) - 0.8\log_2(0.99) \approx 1.34$ bits per day. The excess of about 0.62 bits is $D_{\mathrm{KL}}(\mathrm{Bernoulli}(0.2) \| \mathrm{Bernoulli}(0.01))$. Reversing KL would weight outcomes as if 1% rain were the truth; that is a different evaluation population. The average log-loss comparison still says nothing about the costs of failing to warn on one particular day.

9.1.1 Calibration Is Not the Same as Discrimination

A predictor is calibrated when, among cases assigned probability r to an event, the event occurs about a fraction r of the time, under the population and conditions used to assess calibration. A model can rank cases well while being overconfident, or be calibrated on average while ranking poorly. Log loss is a proper scoring rule: in expectation, reporting the true probability minimises the score. But a single aggregate log-loss value combines calibration, resolution, and irreducible uncertainty; it is not a calibration diagnostic on its own.

Reliability diagrams compare predicted probability bins with observed frequencies, though the binning and sample size matter. For multiclass forecasts, calibration can be defined in several ways (top-label, classwise, or stronger conditional forms); a paper should say which one it evaluates. A model calibrated on one distribution can become miscalibrated after prevalence shift or covariate shift. If deployment decisions have asymmetric costs, threshold selection should use those costs rather than treating 0.5 as universally optimal.

Information-theoretic scoring and calibration are complementary. A lower held-out cross-entropy indicates better average probabilistic predictions on that test distribution, but it does not show that every subgroup is calibrated, that the model is robust to shift, or that a decision based on its outputs improves outcomes. Inspect the relevant subgroup and shift behaviour directly.

9.2 Evidence, Features, and Predictive Information

For a target Y and feature vector X, the mutual information $I(X;Y)$ measures statistical dependence under a joint distribution. If $I(X;Y) = 0$, the variables

are independent in that distribution, so no predictor using X can improve on the target's marginal distribution in expected log loss. If it is positive, some dependence exists, but it does not specify a simple predictive rule, achievable accuracy under a restricted model class, or causal effect.

Feature selection by mutual information can detect non-linear dependence that correlation misses. It can also overstate evidence when many candidate features are screened on the same finite sample. Estimation bias and multiple comparisons compound one another. Use held-out evaluation or nested validation for feature selection, and estimate the relevant information on data not used to choose the feature whenever possible. A high univariate $I(X_j; Y)$ does not imply that feature j adds information conditional on other features; redundant and synergistic features complicate ranking.

For example, if two columns are identical copies of a signal, each has the same marginal MI with the target, but the second adds no conditional information once the first is known: $I(Y; X_2 \mid X_1) = 0$. In the parity construction, by contrast, either input alone has zero MI with the parity target, while both together determine it. A feature-ranking procedure that only scores one column at a time would discard both. The choice between marginal and conditional information is therefore a question about the feature set and prediction task, not an interchangeable implementation detail.

The data-processing inequality gives a useful ceiling. If $Y \to X \to Z$ is a Markov chain, then $I(Y; Z) \leq I(Y; X)$. A learned representation $Z = f(X)$ cannot contain more population information about Y than the original X. Yet a representation can make the relevant information easier for a finite or restricted predictor to use, improve optimisation, reduce noise, or satisfy a resource constraint. The inequality concerns information in the random variables, not the computational accessibility of a useful decision rule.

9.3 The Information Bottleneck

The information bottleneck method frames representation learning as a trade-off. Given input X, target Y, and representation Z with Markov structure $Y \to X \to Z$, one form minimises

$$\mathcal{L}_{\text{IB}} = I(X; Z) - \beta I(Z; Y),$$

where $\beta \geq 0$ controls the trade-off between compressing the input and retaining information about the target. A small first term favours a representation that discards details of X; a large second term rewards predictive dependence with Y. This is an objective defined from a joint distribution, not a universal law of good representations.

The objective depends on how Z is stochastic or deterministic, what distribution is used, how each MI term is estimated, and what value of β is selected. For deterministic neural representations of continuous variables, formal MI can be infinite or ill-behaved under common idealisations; practical variational bounds and noise injections change the estimation problem. Optimising a lower bound does not necessarily optimise the exact objective. Report the estimator and assumptions, and assess downstream performance on held-out data.

An information bottleneck can also discard information needed for a subgroup, a later task, or a shifted distribution, because $I(Z;Y)$ rewards average dependence for the chosen target and population. Choosing the target is therefore consequential. A representation that compresses away a feature correlated with a protected group may still encode that group through other features; low $I(X;Z)$ is not itself a privacy guarantee.

9.3.1 Evidence Is Not Causation

Mutual information is symmetric association. It does not orient an arrow, distinguish confounding from direct influence, or identify what would happen under intervention. A common cause can make X and Y dependent even if changing X would not change Y. Conditioning can create dependence as well as remove it, as in collider structures. Causal conclusions require a causal model, design assumptions, and often experimental or quasi-experimental evidence beyond an observational joint distribution.

The Markov notation $X \to Y \to Z$ in the data-processing inequality denotes conditional independence, not necessarily a causal sequence. It is safe to say that post-processing Y into Z cannot increase information about X when that factorisation holds. It is unsafe to infer from the arrows alone that X caused Y or that the process occurred in time. See Dependence, Conditioning, and Bayes' Rule for this distinction.

A concrete observational counterexample is a common weather variable U that

affects both ice-cream purchases X and swimming visits Y. Their MI can be positive even if buying ice cream has no effect on swimming. Randomising purchases would break the association between X and U while leaving the weather pathway to swimming intact. The joint law of observed (X, Y) alone cannot distinguish this structure from a direct effect of X on Y; both can generate dependence. An intervention claim requires additional assumptions or data.

9.3.2 Privacy: Information Measures and Guarantees

If sensitive data are represented by S and a released output by Z, mutual information $I(S; Z)$ measures average statistical dependence under the chosen joint distribution. It can compare mechanisms under a stated prior and population. It is not, by itself, a complete privacy guarantee: it can average away a severe disclosure affecting a rare person, depends on the prior, and does not state what an adversary knows or can do.

Differential privacy instead bounds how much a mechanism's output distribution can change when one person's data are changed, with a guarantee quantified by parameters and a defined neighbouring-dataset relation. It is designed to hold uniformly across datasets and does not require a prior distribution over sensitive values. This primer need not develop that framework, but should not use "low mutual information" as a synonym for differential privacy. Conversely, a differential privacy guarantee does not promise that a released aggregate is useful for every analysis; utility remains a separate question.

The data-processing inequality says that post-processing a release cannot increase its mutual information with the secret. It also cannot invalidate a formal differential-privacy guarantee when the post-processing uses no additional private data. These are related but distinct statements with different premises and threat models. Reconstruction, membership inference, and attribute inference depend on available auxiliary data, the adversary, and the output interface.

Consider a release that reveals a sensitive bit only for one rare subgroup and is constant for everyone else. If that subgroup has tiny probability in the chosen population, the average $I(S; Z)$ can be small while disclosure for a member of the subgroup is complete. Changing the prior changes the average again. This does not make MI useless: it is a coherent population dependence measure and can bound some average inference quantities under explicit assumptions.

It does show why an application with individual-level promises needs a threat model and a guarantee that addresses the relevant worst cases or tail events.

9.4 A Practical Quantity-Selection Guide

A dataset can support several legitimate questions, each with a different information quantity. Before calculating, identify the random variables, population, and decision. Then choose a quantity whose units and assumptions match that question.

Practical question	Start with	State explicitly	Do not infer automatically
How uncertain is one source symbol under this model?	$H(X)$	Source distribution, alphabet, bits per symbol	Semantic value or file size
How many bits does a probabilistic predictor spend on held-out outcomes?	Mean log loss / cross-entropy	Outcome model, test distribution, handling of zero probabilities	Calibration or causal validity
How dependent are two measured variables?	$I(X;Y)$	Joint population, variable types, estimator, uncertainty	Direction or intervention effect
How much can a chosen channel input tell the receiver?	$I_p(X;Y)$	Channel law and the actual input law	Maximum channel capability
What is the ideal reliable rate of a modelled channel?	Capacity C	Channel assumptions, allowed inputs, bits per use	Finite-code throughput

Practical question	Start with	State explicitly	Do not infer automatically
What rate permits an acceptable average reconstruction?	$R(D)$	Source model, distortion function, expected-loss target	Per-item or perceptual guarantee
Does a learned representation retain target-relevant dependence?	$I(Z;Y)$ and a predictive evaluation	Joint distribution, estimator, representation and target	Generalisation or fairness
Is sensitive information exposed by a release?	Threat model plus an appropriate privacy definition	Adversary, auxiliary knowledge, adjacency or prior	Privacy from MI alone

A disciplined analysis often reports more than one quantity. For example, a communications design may report capacity as a theoretical ceiling, code rate and block error as an implementation result, and delivered application throughput after overhead. A compression study may report bitrate and a distortion metric, plus task performance if users make decisions from reconstructed data. Keep the model bound separate from the empirical outcome.

Here is the bookkeeping for a simple end-to-end link. Suppose a source produces 12 symbols per second, a source coder averages 1.4 bits per source symbol, and the resulting bitstream is protected by a channel code of rate $r_c = 0.7$ information bits per transmitted channel bit. The physical channel must carry $12(1.4)/0.7 = 24$ channel bits per second before framing. If the channel offers 30 uses per second and has capacity 0.9 bits per use, its ideal information ceiling is 27 bits per second. The fact that 24 is below 27 is a necessary asymptotic compatibility check under the models; it does not guarantee the chosen finite source coder, channel code, or latency meets requirements. If the source coder's 1.4-bit figure is a finite-file average including headers, it should not be compared casually with a stationary entropy rate either.

This bookkeeping also catches a common category error: dividing a source en-

tropy in bits per symbol directly by a channel capacity in bits per use yields no meaningful utilisation until the source symbol rate and channel-use rate are included. A full system model may also include packet headers, retransmissions, idle time, and unequal error protection. Add these as explicit rate factors rather than quietly folding them into “capacity”.

10 Exercises

Unless stated otherwise, logarithms are base 2. Work from the stated model and check the units. Outline solutions follow each prompt.

10.1 Exercise 1: Probability and Labelling

A three-outcome variable has probabilities $1/2, 1/3, 1/6$. Verify normalisation and calculate the expectation of the numeric labels $0, 1, 2$.

Outline Solution. The probabilities sum to 1. The expectation is $0(1/2) + 1(1/3) + 2(1/6) = 2/3$. Relabelling the outcomes changes this expectation, unlike entropy, which depends only on the probability multiset.

10.2 Exercise 2: KL Direction and Support

Let $P = (1, 0)$ and $Q = (1/2, 1/2)$. Find both directed KL divergences.

Outline Solution. $D_{\mathrm{KL}}(P\|Q) = 1$ bit. $D_{\mathrm{KL}}(Q\|P) = +\infty$ because Q assigns positive mass where P assigns zero. This also demonstrates asymmetry and the support rule.

10.3 Exercise 3: Pairwise Independence and Parity

In the parity construction with independent fair X, Y and $Z = X \oplus Y$, show that each pair is independent but $H(Z \mid X, Y) = 0$.

Outline Solution. For any fixed value of one bit, XOR with the other fair independent bit is fair, giving pairwise independence. Once both inputs are given, the output is fixed.

10.4 Exercise 4: Equality in Data Processing

A system has $X \to Y \to Z$ and a deterministic $Z = f(Y)$. Does this imply $I(X;Z) = I(X;Y)$?

Outline Solution. No. DPI gives only the upper bound. Equality requires that Z preserve all information in Y relevant to X, equivalently $I(X;Y \mid Z) = 0$.

10.5 Exercise 5: Kraft and a Code Assignment

A binary prefix code has lengths 1, 2, and 2. Is such a code possible? Give one assignment.

Outline Solution. Kraft's sum is $1/2 + 1/4 + 1/4 = 1$, so it is possible. Codewords `0`, `10`, and `11` work.

10.6 Exercise 6: Fractional Entropy and Integer Lengths

Explain why a 1.846-bit entropy does not imply that a one-symbol binary code can give every symbol a codeword of length 1.846 bits.

Outline Solution. Binary codeword lengths are non-negative integers. Entropy is an expected lower bound; block codes and arithmetic coding can amortise fractional information over long sequences.

State the assumptions alongside each calculation; a numerical answer alone does not identify the model.

10.7 Exercise 7: Input Bias on a BSC

A BSC has crossover probability $\varepsilon = 0.15$. The input has $P(X = 1) = 0.2$.

1. Find $P(Y = 1)$.
2. Compute $I(X;Y)$ using $h_2(P(Y = 1)) - h_2(\varepsilon)$.
3. Compute the channel capacity and explain why it is not the value in part 2.
4. What changes if $\varepsilon = 1$?

Outline Solution. $P(Y = 1) = 0.15 + 0.2(1 - 0.30) = 0.29$. Thus $I = h_2(0.29) - h_2(0.15) \approx 0.869 - 0.610 = 0.259$ bits per use. Capacity is $1 - h_2(0.15) \approx 0.390$ bits per use, achieved by a uniform input, not the specified biased input. At $\varepsilon = 1$, the channel flips deterministically; $I(X;Y) = H(X)$ for

any input law and capacity is one bit per use. A deterministic flip is invertible, not destructive noise.

10.8 Exercise 8: Erasures and Substitutions

A BEC has erasure probability 0.12. A second channel is a BSC with crossover probability 0.12.

1. Find each capacity.
2. Explain the role of the receiver's knowledge of erased positions.
3. Are these channels equally difficult for every code and block length?

Outline Solution. BEC capacity is $1 - 0.12 = 0.88$ bits per use. BSC capacity is $1 - h_2(0.12) \approx 0.471$ bits per use. The BEC reveals which symbols are unavailable; the BSC output can be a plausible but incorrect bit. The capacities compare asymptotic optimal rates under their respective models. They do not imply equal finite-block error, decoder complexity, or performance for a particular code. The numerical equality of the error parameter has no common operational meaning.

10.9 Exercise 9: A Z-Channel Optimisation

Input 0 is always received as 0. Input 1 is received as 0 with probability $q = 0.1$ and as 1 otherwise.

1. Write $I_a(X;Y)$ when $P(X = 1) = a$.
2. Explain why $a = 1/2$ is not generally optimal.
3. Find the derivative condition for an interior maximiser.

Outline Solution. $P(Y = 1) = 0.9a$ and $H(Y \mid X) = ah_2(0.1)$, so $I_a = h_2(0.9a) - ah_2(0.1)$. The conditional output entropy differs by input row, so a uniform input need not maximise the output-information balance. Since $h_2'(u) = \log_2((1-u)/u)$, the stationary condition is $0.9 \log_2((1-0.9a)/(0.9a)) = h_2(0.1)$. Verify the candidate is in $[0, 1]$ and compare boundaries if solving numerically.

10.10 Exercise 10: Coding Rate and Error Probability

For threefold repetition on a BSC with $\varepsilon = 0.05$, use majority decoding.

1. Find the code rate.
2. Calculate the block error probability.

3. Compare the result with the BSC capacity, and state why this does not verify the coding theorem.

Outline Solution. One information bit uses three channel symbols, so the rate is $1/3$ bit per use. Failure requires at least two flips: $3(0.05)^2(0.95) + (0.05)^3 = 0.00725$. Capacity is $1 - h_2(0.05) \approx 0.714$ bits per use. The code's rate is below capacity and has a small but non-zero error for this one block length. The theorem concerns sequences of codes as block length tends to infinity, not this particular code.

10.11 Exercise 11: Choose a Distortion Measure

A logger stores a binary warning flag with prevalence $P(X = 1) = 0.03$. Compare the distortions (a) Hamming loss and (b) asymmetric loss where missing a warning costs 20 and a false warning costs 1. What is the zero-rate constant reconstruction under each expected-distortion criterion? What does that tell you about the objective?

Outline Solution. Under Hamming loss, always output 0 gives expected distortion 0.03, while always output 1 gives 0.97; the zero-rate choice is 0. Under asymmetric loss, always output 0 costs $20(0.03) = 0.6$, while always output 1 costs $1(0.97) = 0.97$, so output 0 remains preferable here. If prevalence or costs change, the choice can change. The example makes clear that "distortion" encodes application costs; a rate–distortion answer is only as appropriate as that cost model.

10.12 Exercise 12: Gaussian Rate–Distortion

An iid Gaussian source has variance $\sigma^2 = 9$ and squared-error distortion is allowed up to $D = 1$ per sample.

1. Find the asymptotic rate–distortion bound.
2. What rate suffices for $D \geq 9$?
3. Why can the rate not be read as an exact finite-file size?

Outline Solution. $R(1) = \frac{1}{2}\log_2(9) \approx 1.585$ bits per sample. For $D \geq 9$, outputting the mean zero requires no bits and meets the expected distortion target. The theorem is asymptotic for iid Gaussian samples under squared error; finite block coding, overhead, and a particular sample realisation affect actual file length and distortion.

10.13 Exercise 13: Entropy Bias and a Sparse Table

You observe 25 pairs of categorical variables, each with 10 possible categories, but only 13 joint cells are occupied. The plug-in mutual information is 0.42 bits.

1. Is 0.42 an unbiased estimate of population MI?
2. Name two useful diagnostics or uncertainty procedures.
3. Explain why permuting one variable answers a different question from bootstrapping paired observations.

Outline Solution. No. Plug-in MI is generally positively biased under independence, particularly with sparse tables and many possible cells. Report support and cell counts, compare against a permutation null if pairings are exchangeable, and bootstrap pairs or clusters to assess sampling variation. Permutation estimates a reference distribution under a null of no association while retaining marginals; bootstrap estimates sampling variability around the observed joint distribution. Neither procedure repairs absent support automatically.

10.14 Exercise 14: KL Direction and Prediction

The true outcome distribution is $p = (0.8, 0.2)$ and a model predicts $q = (0.6, 0.4)$.

1. Write the model's expected log loss.
2. Decompose it into entropy and KL divergence with the correct direction.
3. What happens if the model assigns $q_2 = 0$?

Outline Solution. Expected log loss is $-0.8 \log_2(0.6) - 0.2 \log_2(0.4) = H(p) + D_{\mathrm{KL}}(p \| q)$. The true distribution comes first. If the second outcome has positive true probability but $q_2 = 0$, expected log loss and the KL divergence are infinite. A finite sample might fail to expose this failure, which is why unsmoothed zero predictions are dangerous.

10.15 Exercise 15: Information Bottleneck and Causation

A representation $Z = f(X)$ has high estimated $I(Z; Y)$ on training data. A team concludes that Z causes Y and that the representation preserves every important input property.

Identify two errors and propose two checks.

Outline Solution. MI is symmetric association, not causal direction; confounding or selection can produce the dependence. High $I(Z;Y)$ says nothing about information discarded from X that matters for another task or subgroup, and a training estimate can be biased or overfit. Use an identified causal design for causal claims, estimate performance and MI on held-out data with uncertainty, and audit subgroup and secondary-task behaviour. State the estimator and target population.

10.16 Exercise 16: Select a Quantity for a System Claim

A team reports: "Our sensor system has capacity 2 bits/s and therefore loses no more than 1% of measurements." The model is a BSC used 4 times per second with crossover probability 0.1. The team also compresses a continuous sensor value before transmission.

Rewrite the claim as separate, meaningful quantities and identify missing information.

Outline Solution. The BSC capacity is approximately 0.531 bits per channel use, so the ideal asymptotic rate is about $4(0.531) = 2.12$ bits/s under the stated model, before overhead. This says nothing about a particular code's finite-block error, and the BSC crossover is not a measurement-distortion percentage after compression. To bound reconstruction loss, specify the source law, distortion measure, target average or maximum error, source coding method, channel code, block length, and end-to-end evaluation. Keep source symbols per second, channel uses per second, bits per use, and bits per second distinct.

11 Glossary

These short definitions point to the first full treatment. In each application, the units and distributional assumptions belong with the reported value.

Term	Definition	First Treatment
Achieved mutual information	$I_p(X;Y)$ for a specified input distribution p and channel law	Mutual Information

Term	Definition	First Treatment
Alphabet	Possible symbols of a discrete random variable or code	Sample Spaces and Random Variables
Asymmetric channel	A channel whose transition behaviour is not invariant under relabelling or symmetry of its inputs and outputs	Asymmetric Channels and Input Optimisation
Average error probability	Probability of decoding the wrong message, averaged over the stated message distribution and channel noise	Capacity and the Channel Coding Theorem
Binary erasure channel (BEC)	Binary-input channel that replaces a symbol with a visible erasure marker with a stated probability	Binary Erasure Channel
Binary symmetric channel (BSC)	Binary-input channel that flips each bit independently with a fixed crossover probability	Binary Symmetric Channel: One Input Law, One Capacity
Capacity	Maximum mutual information over allowed input distributions; for a DMC, measured in bits per channel use	Capacity and the Channel Coding Theorem
Channel coding	Adding structured redundancy to improve recovery after channel errors	Capacity and the Channel Coding Theorem
Conditional entropy	Expected uncertainty in one variable after observing another	Conditional Entropy as Remaining Uncertainty

Term	Definition	First Treatment
Cross-entropy	Expected negative log probability assigned by one distribution to outcomes generated under another	Cross-Entropy and Log Loss
Data-processing inequality	Markov-chain bound saying that processing an observation cannot increase its mutual information with a specified source	Data-Processing Inequality
Differential entropy	Continuous-density analogue of entropy, which depends on coordinates and can be negative	Continuous Entropy and Relative Entropy
Distortion measure	A non-negative cost assigned to a source value and its reconstruction	Distortion and the Rate–Distortion Function
Discrete memoryless channel (DMC)	Finite-alphabet channel with a fixed transition law and conditionally independent outputs across uses	A Channel as a Conditional Distribution
Empirical distribution	Observed relative frequencies used to approximate an unknown population distribution	Plug-In Estimates and Their Bias
Entropy	Expected surprisal under a discrete source law	Shannon Entropy

Term	Definition	First Treatment
Erasure	Missing channel output whose position is explicitly known to the receiver	Binary Erasure Channel
Hamming distortion	Zero cost for an exact match and unit cost for a mismatched symbol	Binary Hamming Distortion
Huffman code	Minimum-expected-length prefix code for a known finite source encoded one symbol at a time	Huffman Coding and a Complete Example
Information bottleneck	Objective trading off retained information about an input against predictive information about a target	The Information Bottleneck
KL divergence	Directed expected log likelihood ratio, or excess expected log loss under a mismatched model	KL Divergence
Mutual information	Average reduction in uncertainty about one variable supplied by observing another	Mutual Information
Miller–Madow correction	First-order correction to plug-in entropy bias for a fixed finite support under regular large-sample conditions	Plug-In Estimates and Their Bias

Term	Definition	First Treatment
Permutation test	Null procedure that reassigns labels under an exchangeability assumption to estimate a test-statistic reference distribution	Permutation Tests for Dependence
Prefix code	A code in which no complete codeword begins another codeword	From Probabilities to Code Lengths
Rate–distortion function	Minimum mutual information needed to meet an expected-distortion constraint	Distortion and the Rate–Distortion Function
Sampling zero	Unobserved category or joint cell that may nevertheless have positive population probability	Joint Tables, Sparse Cells, and Mutual Information
Source coding	Representing a source with fewer expected bits by exploiting a known distribution or dependence	Lossless Source Coding
Structural zero	Category or cell that is impossible under the stated design or model	Joint Tables, Sparse Cells, and Mutual Information
Surprisal	Negative log probability assigned to an outcome under a specified law	Surprise as Log Probability

Term	Definition	First Treatment
Typicality	A block-level property meaning empirical frequencies or information densities are close to their model values	Capacity and the Channel Coding Theorem

12 Further Reading

The following sources support the main definitions and theorems. The annotations indicate where to continue after the primer; they do not replace the stated hypotheses of a result.

- Claude E. Shannon, “A Mathematical Theory of Communication”, Part I and Part II, *Bell System Technical Journal* 27 (1948). The original framework for entropy, communication, and channel coding; modern texts provide the theorem conventions used here.
- Thomas M. Cover and Joy A. Thomas, *Elements of Information Theory*, second edition, Wiley (2006). A systematic treatment of entropy identities, coding theorems, Gaussian channels, and rate-distortion, with complete proofs beyond the sketches here.
- David J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*, Cambridge University Press (2003). The publisher’s excerpt introduces communication and inference; the full book develops them with computational exercises.
- David A. Huffman, “A Method for the Construction of Minimum-Redundancy Codes”, *Proceedings of the IRE* 40 (1952), 1098–1101. The original binary prefix-code construction.
- Imre Csiszár and János Körner, *Information Theory: Coding Theorems for Discrete Memoryless Systems*, second edition, Cambridge University Press (2011). A deeper reference for discrete coding-theorem proofs and conventions.
- Claude E. Shannon, “Coding Theorems for a Discrete Source With a Fidelity Criterion”, *IRE National Convention Record* 7, Part 4 (1959), 142–163. The foundational source for coding under a fidelity or distortion criterion; the

link is a scan of the original paper.

- Liam Paninski, “Estimation of Entropy and Mutual Information”, *Neural Computation* 15 (2003), 1191–1253. Analyses bias, consistency, and finite-sample limits of information estimators.
- Toby Berger, “Rate-Distortion Theory”, *Encyclopedia of Telecommunications* (2003). A concise account of lossy coding and the relationship between source and channel models.